



Algorithm Complexity: An Introduction



At the end of this lesson, you should be able to:

- State the concept of time complexity in algorithm implementation
- Explain the Big O notation
- Describe how Big O can be used to compare time complexity of algorithms



Recall

Algorithms

- step-by-step instructions given to computer on how to solve a given problem



- multiple possible algorithms as well as implementations
- how to compare them?
- how to evaluate which is the 'better' algorithm?



Comparison in terms of execution time



Shorter execution time $\Rightarrow$ better?

But execution time depends on many factors

- the speed of the computer
- the way the algorithm is implemented
 - pre-compute lookup table
 - loop unrolling technique
 - input data value and input data size

Instruction Steps

Count the number of instructions it takes to execute the algorithm



- independent of the computer
- more steps $\Rightarrow$ longer execution time

But the number of steps may still depend on the data involved in the computation

Example

```
def linearSearch(List, item_x):  
    for i in range(len(List)):  
        if List[i] == item_x:  
            return i  
    return -1
```


Asymptotic Behavior

More important to consider the worst case situation

- when item_x is not in the List
- number of instruction steps will be the most

As the number of entries in the List increases

- number of steps under worst case situation also increases

Note that

- 'input' data size $\equiv$ number of entries in the List

In time complexity analysis

- growth pattern of the number of steps as input data size increases indefinitely
- asymptotic behavior of running time – Big O

Example

```
def linearSearch(List, item_x):  
    for i in range(len(List)):  
        if List[i] == item_x:  
            return i  
    return -1
```

BIG O Notation

- measure and compare the time complexity of algorithms
- pattern of execution time of algorithm as input data size grows

Execution Time

- in terms of the number of instruction steps
- for worst case situation

Big O gives an upper bound on the asymptotic growth of an algorithm

Analysis of a Linear Search Algorithm

Assumption

- each line of the code statement can be executed in one step

Worst case

- item_x not in the list (of length n)
- $T(n) = 2 + 4n + 1 = 3 + 4n$

$$T(n) = 3 + 4n$$

n = 'Input' Data size $\equiv$ number of entries in the List

Example

```
def linearSearch(List, item_x):
    max_pos = len(List)
    i = 0
    while i < max_pos:
        if List[i] == item_x:
            return i
        i = i + 1
    if i == max_pos:
        return - 1
```

1
1
n times
1 x n
-
1 x n
1 x n
1

Growth Order of $T(n)$

$$T(n) = 3 + 4n$$

- minimum value of $T = 3$
- when $n \gg 3$, $T \rightarrow 4n$
 - e.g. for $n = 1000$

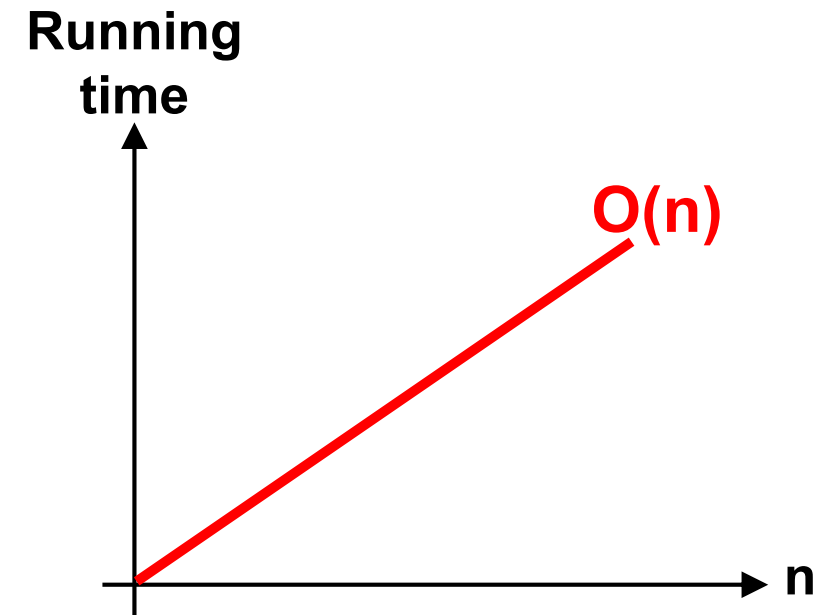
$$T = 3 + 4000 = 4003 \approx 4000$$
- $T(n) \approx 4n$ for large n

Asymptotic behavior

- $T(n)$ increases proportionally with n
 - e.g. $T(n)$ doubles when n is doubled
- Growth order: $f(n) = n$

Complexity using Big O notation: $O(f(n)) = O(n)$

- Linear complexity



Program Complexity Types

Common Types of Complexity

$O(1)$

Constant complexity, where $f(n) = 1$

$O(n)$

Linear complexity, where $f(n) = n$

$O(\log n)$

Logarithmic complexity, where $f(n) = \log n$

$O(n^k)$

Polynomial complexity, where $f(n) = n^k$, with $k = \text{constant}$
e.g. $O(n^2)$ = Quadratic complexity

$O(K^n)$

Exponential complexity, where $f(n) = k^n$

Constant Complexity $O(1)$

$O(1) \equiv$ Constant Complexity

Algorithm always uses the same amount of time to execute for all inputs

Example:

*Pre-compute lookup
table-based algorithm*

Example 1

```
def lookup(position)
    return
lookupTable(position)
```

$$T(n) = 1 \equiv 1.1 \Rightarrow f(n) = 1;$$

$$O(fn) = O(1)$$

Example 2

```
def lookup(position)
    print("Entry position = " , position)
    return lookupTable(position)
```

$$T(n) = 2 \equiv 2.1 \Rightarrow f(n) = 1; O(fn) = O(1)$$

Running
time



Linear Complexity $O(n)$

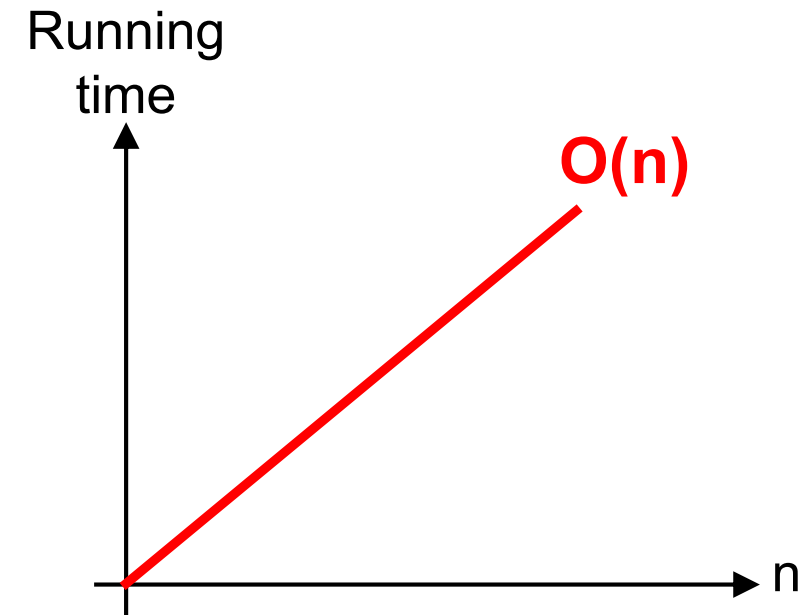
$O(n) \equiv$ Linear Complexity

Algorithm execution time increases linearly in proportion with (input) data size

Example

```
def linearSearch(List, item_x):
    for i in range(len(List)):
        if List[i] == item_x:
            return i
    return -1
```

$f(n) = \text{len(List)}$ $O(fn) = O(\text{len(List)}) = O(n)$



Polynomial Complexity $O(n^K)$

$O(n^K) \equiv$ Polynomial Complexity

Occurs for algorithm that contains nested loops

Example: Quadratic complexity $O(n^2)$

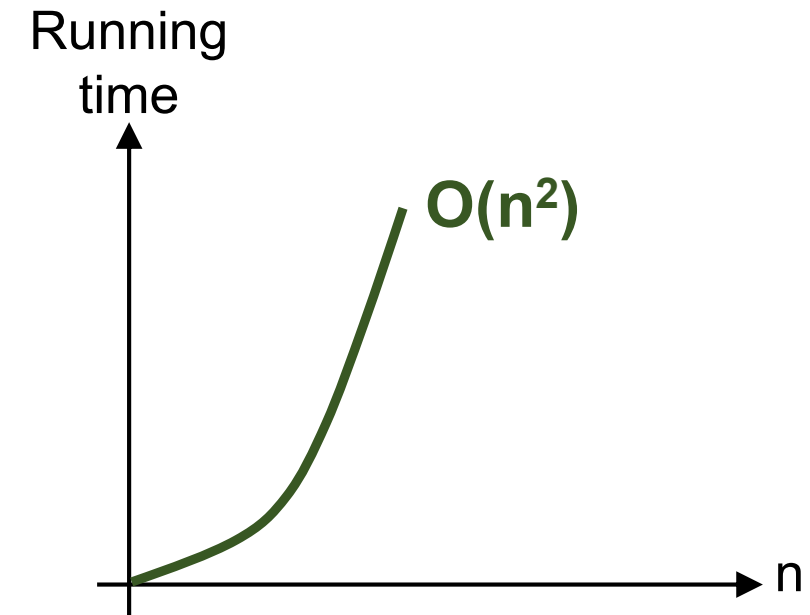
- execution time is proportional to the square of the input data size

Example: Quadratic Complexity $O(n^2)$

```
def checkDuplicate(List1, List2):
    for i in range(len(List1)):
        for j in range(len(List2)):
            if List1[i] == List2[j]:
                return List1[i]
    return -1
```

$f_{in}(n) = \text{len(List2)}$; $f_{out}(n) = \text{len(List2)}$

$O(fn) = O(f_{in}(n) \cdot f_{out}(n)) \equiv O(n \cdot n) = O(n^2)$

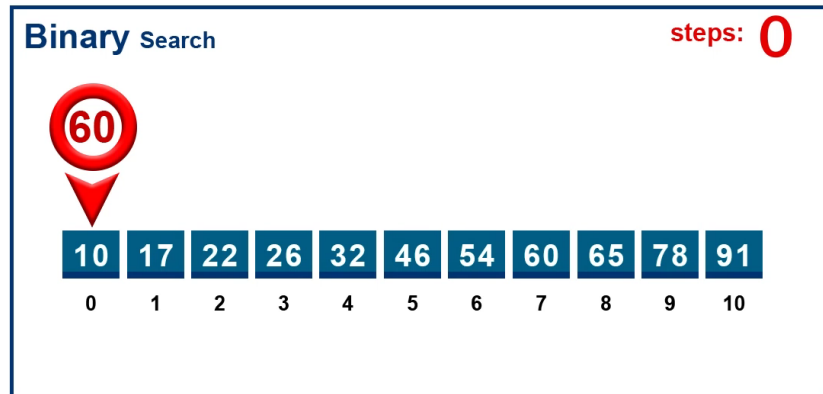


Logarithmic Complexity $O(\log n)$

$O(\log n) \equiv$ Logarithmic Complexity

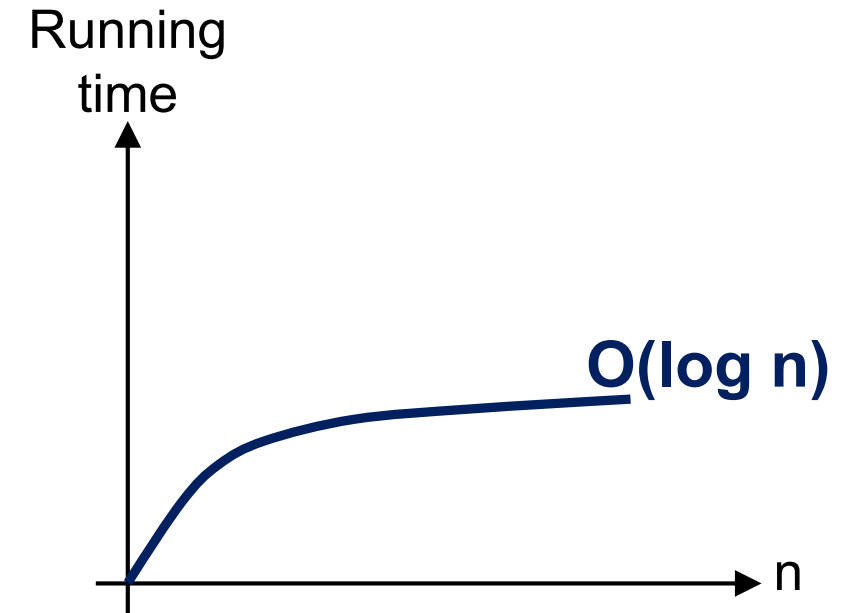
Execution time grows as the log of input

Example: Binary Search



$$(n) = \log_2(\text{len}(\text{List}))$$

$$O(\text{fn}) = O(\log_2(\text{len}(\text{List}))) \equiv O(\log n)$$



Exponential Complexity $O(k^n)$

$O(k^n) \equiv$ Exponential Complexity

Occurs for algorithm that contains recursive call

Example:

Fibonacci sequence computation

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

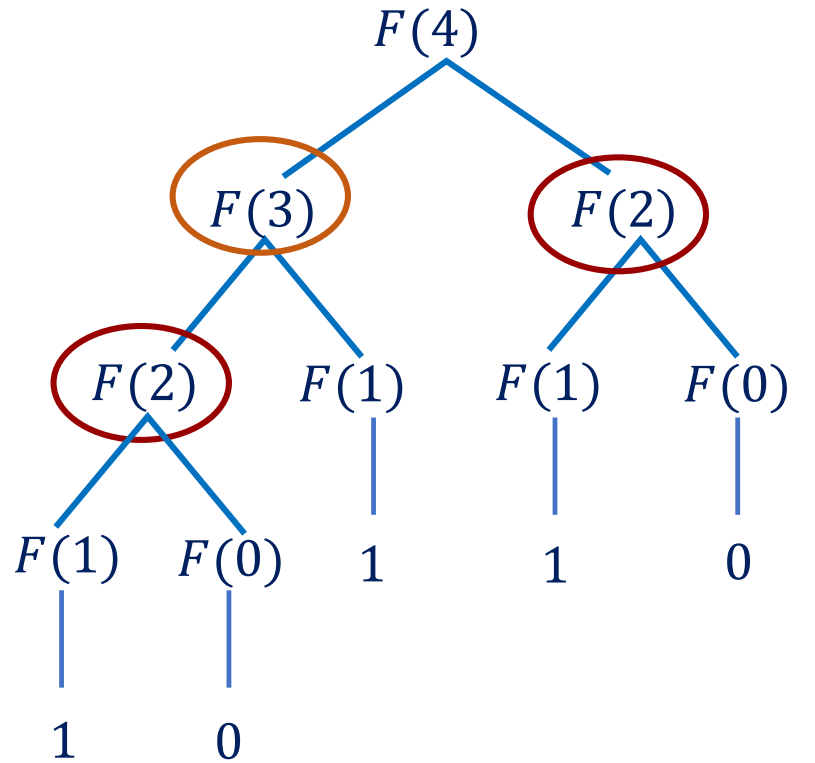
Each number can be derived based on the formula:

$$F(n) = F(n-1) + F(n-2), n \geq 2$$

Example

```
def rFib (n):
    if (n == 0):
        return 0
    elif (n == 1):
        return 1
    elif (n > 1):
        return (rFib(n-1) + rFib(n-2))
    else:
        return -1
```

Recursion Fibonacci



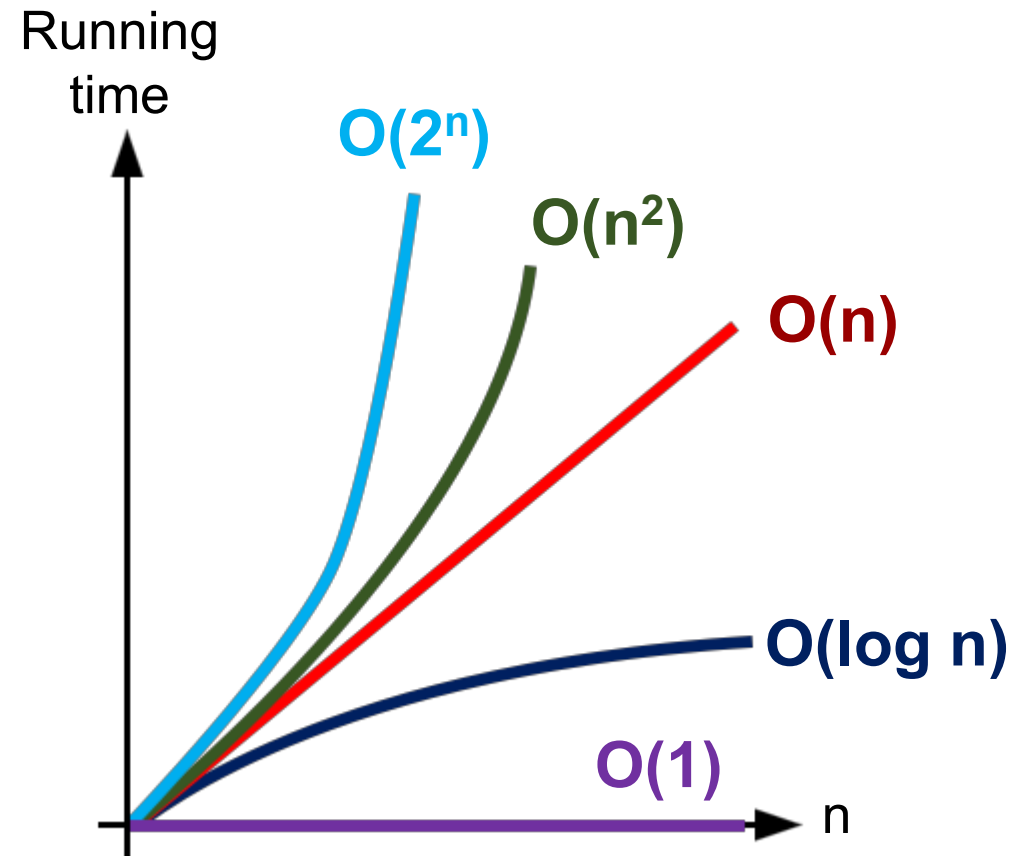
$$f(n) = 2^n$$

$$O(f(n)) = O(2^n)$$

Example

```
def rFib (n):  
    if (n == 0):  
        return 0  
    elif (n == 1):  
        return 1  
    elif (n > 1):  
        return (rFib(n-1) + rFib(n-2))  
    else:  
        return -1
```

Rate of Growth



Iteration Fibonacci

An alternative algorithm for Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

Computation based on iteration:

$$F(n) = F(n - 1) + F(n - 2), n \geq 2$$

```
def iFib(n):
    if (n == 0):
        return 0
    elif (n == 1):
        return 1
    elif (n == 2):
        return 1
    elif: # n > 2
        #continue in next
```

```
elif : # n > 2
    fn0 = 0
    fn1 = 1
    fn2 = 1
    for i in range(n-2):
        fn0 = fn1
        fn1 = fn2
        fn2 = fn0+fn1
    return fn2
```

$$T(n) = 4n + 4$$

$$f(n) = n$$

$$\Rightarrow O(n)$$

i.e. Linear Complexity

Iteration vs. Recursion Fibonacci Computation

<pre>*Python 3.7.0 Shell* File Edit Shell Debug Options Window Help Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32 Type "copyright", "credits" or "license()" for more information. >>> ===== RESTART: C:\Users\russell\Downloads\iFibonacci.py ===== Iteration Fibonacci - Input a number:</pre>	<pre>*Python 3.7.0 Shell* File Edit Shell Debug Options Window Help Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32 Type "copyright", "credits" or "license()" for more information. >>> ===== RESTART: C:\Users\russell\Downloads\rFibonacci.py ===== Recursion Fibonacci - Input a number:</pre>
<pre>*Python 3.7.0 Shell* File Edit Shell Debug Options Window Help Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32 Type "copyright", "credits" or "license()" for more information. >>> ===== RESTART: C:\Users\russell\Downloads\iFibonacci.py ===== Iteration Fibonacci - Input a number:</pre>	<pre>*Python 3.7.0 Shell* File Edit Shell Debug Options Window Help Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32 Type "copyright", "credits" or "license()" for more information. >>> ===== RESTART: C:\Users\russell\Downloads\rFibonacci.py ===== Recursion Fibonacci - Input a number:</pre>

Iteration vs. Recursion Fibonacci Computation (Cont'd)

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\IFibonacci.py =====
Iteration Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\rfibonacci.py =====
Recursion Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\IFibonacci.py =====
Iteration Fibonacci - Input a number:
```

```
Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27 2018, 04:06:47) [MSC v.1914 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\russell\Downloads\rfibonacci.py =====
Recursion Fibonacci - Input a number:
```

Algorithm complexity can be analyzed

- time complexity

Big O

- worst case analysis
- order of growth pattern as input size grows

Asymptotic behavior of algorithms

- useful for comparing and classifying algorithms

Different algorithms for same problem

- compared based on Big O

References for Images

No.	Slide No.	Image	Reference
1	4		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
2	4		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .
3	5		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/stopwatch-timer-watch-seconds-34107/ .



Algorithm Design: Searching

Searching

Search Results in New CX1003 > quick check

 quick check 2 - line 1 and 2 -remove.py	E:\LF work\LF current Courses\New CX1003\Content\...	Type: Python File	Date modified: 6/22/2018 12:34 PM Size: 227 bytes
 quick check 2 - line 1 and 2 -remove.py	E:\LF work\LF current Courses\New CX1003\TEL\3E2	Type: Python File	Date modified: 6/22/2018 12:34 PM Size: 227 bytes
 quick check 2 - line 1 and 2.py	E:\LF work\LF current Courses\New CX1003\Content\...	Type: Python File	Date modified: 6/22/2018 12:34 PM Size: 279 bytes
 quick check 2 - line 1 and 2.py	E:\LF work\LF current Courses\New CX1003\TEL\3E2	Type: Python File	Date modified: 6/22/2018 12:34 PM Size: 279 bytes
 quick check 1.py	E:\LF work\LF current Courses\New CX1003\Content\...	Type: Python File	Date modified: 6/5/2018 10:55 PM Size: 85 bytes
 quick check 1.py	E:\LF work\LF current Courses\New CX1003\TEL\3E2	Type: Python File	Date modified: 6/5/2018 10:55 PM Size: 85 bytes
 quick check 2.py	E:\LF work\LF current Courses\New CX1003\Content\...	Type: Python File	Date modified: 2/16/2018 4:13 PM Size: 253 bytes
 quick check 2.py	E:\LF work\LF current Courses\New CX1003\Content\...	Type: Python File	Date modified: 2/16/2018 4:13 PM

Hard Drive (E:)
Hard Drive (E:)

 searching algorithms 

[All](#) [Images](#) [Videos](#) [News](#) [Maps](#) [More](#) [Settings](#) [Tools](#)

About 191,000,000 results (0.35 seconds)

[Algorithms - Searching Algorithms - GeeksforGeeks](#)
<https://www.geeksforgeeks.org/searching-algorithms/> ▼
Searching Algorithms; Comparisons. Library Implementations of Searching Algorithms; Coding Problems; Quick Links. Searching Algorithms : Linear Search ...
Category Archives: Searching · Linear Search · Binary Search · Jump Search

[Search algorithm - Wikipedia](#)
https://en.wikipedia.org/wiki/Search_algorithm ▼
In computer science, a search algorithm is any algorithm which solves the search problem, namely, to retrieve information stored within some data structure, ...
[Classes](#) · [For virtual search spaces](#) · [For sub-structures of a ...](#) · [References](#)





At the end of this lesson, you should be able to:

- Describe the process of searching
- Explain the importance of different types of searching algorithms
- Search for a given value in an array using linear search and binary search
- Apply search algorithms in problem solving
- Recognize that “no single” best search algorithm applies to all scenarios

Topic Outline



Searching Pizza Hut in North Spine Plaza

- Starts at the first item
- Is it the one I am looking for?
- If not, go to next item
- Repeats until found or all the items are checked

  in North Spine Plaza?

List of Food & Beverage in North Spine

Bakery Cuisine

Subway

Peach Garden Chinese Restaurant

Mr Bean

Pizza Hut

FOUND

The Soup Spoon Union

North Spine Food Court

- Iterates over the sequence, one item at a time, until the specific item is found or all items have been examined
 - The element that needs to be found is called a search key
- Linear search/ sequential search
 - Intuitive approach
 - Starts at the first item
 - Is it the one I am looking for?
 - If not, go to next item
 - Repeats until found or all the items are checked
- This approach is necessary if items are not sorted

Searching in a Sorted List



in North Spine Plaza?

List of Food & Beverage in North Spine



Bakery Cuisine

Mr Bean

North Spine Food Court

Peach Garden Chinese Restaurant

Pizza Hut

Subway

The Soup Spoon Union

NOT HERE

If items are sorted

Searching in a Sorted List (Cont'd)



in North Spine Plaza?

List of Food & Beverage in North Spine



Bakery Cuisine

Mr Bean

North Spine Food Court

Peach Garden Chinese Restaurant

Pizza Hut

Subway

The Soup Spoon Union

NOT HERE

If items are sorted

- Given a list of data, **searching** is finding the location of a particular value or reporting that the value is not present.
- It is one of the fundamental problems in computer science and programming.
- Sorting is done to make searching easier.
- There are multiple searching algorithms to solve problems.
 - How do we know which algorithm is better?

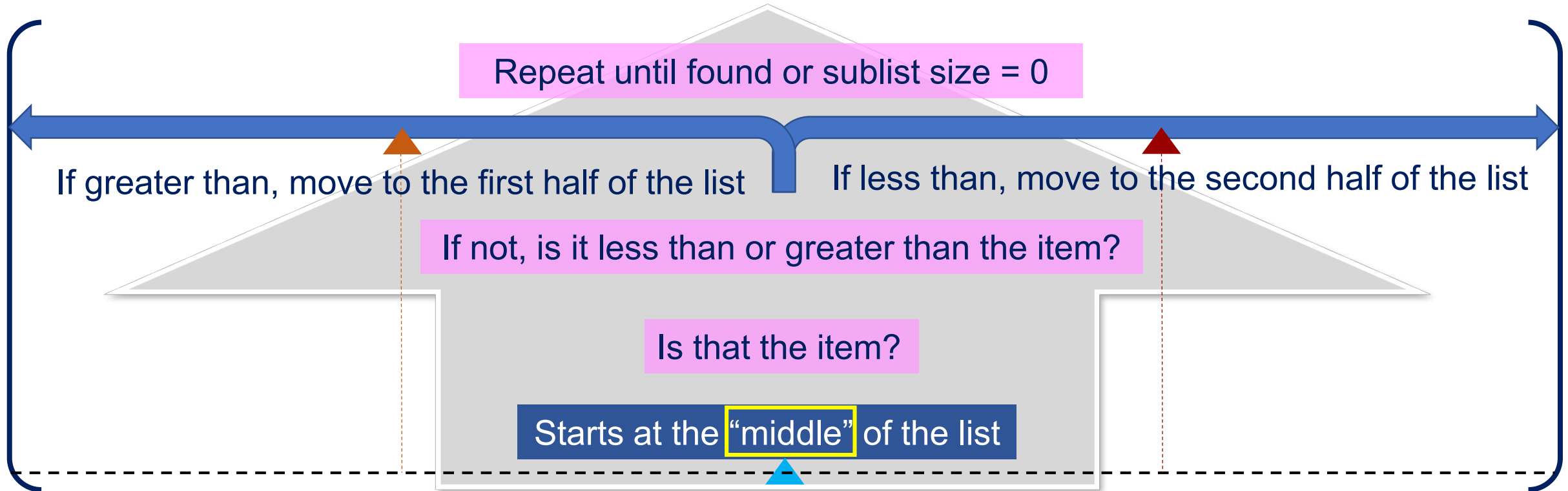
Searching in a Dictionary



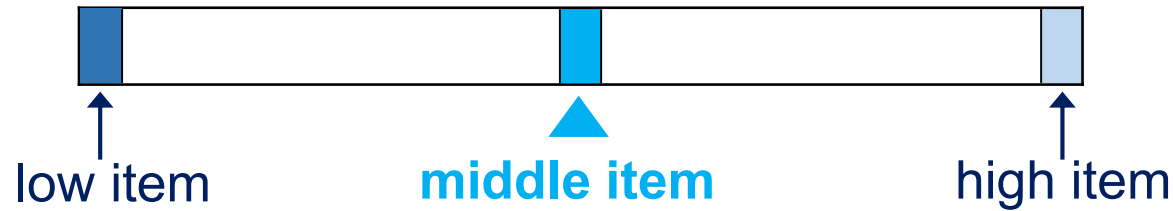
Binary Search on List of Ascending Order

- If items are sorted then you can **divide-and-conquer**
- *dividing your work in half with each step*

Generally a good thing!



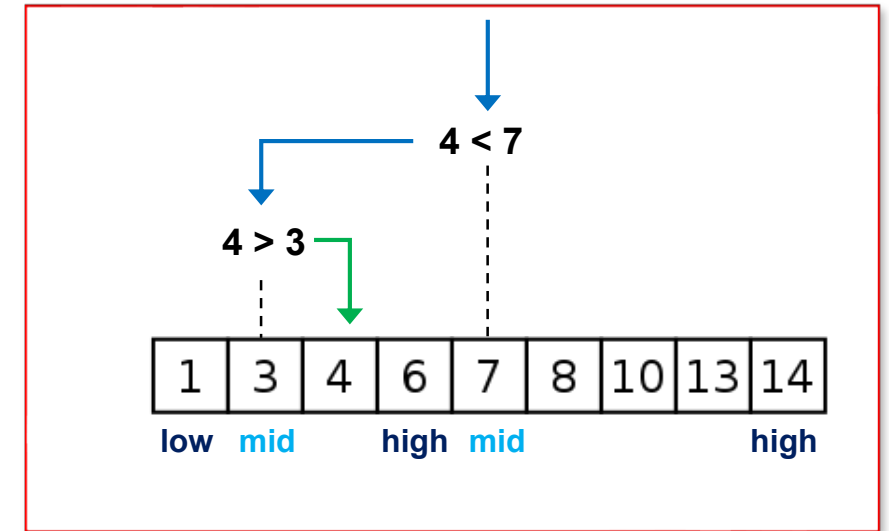
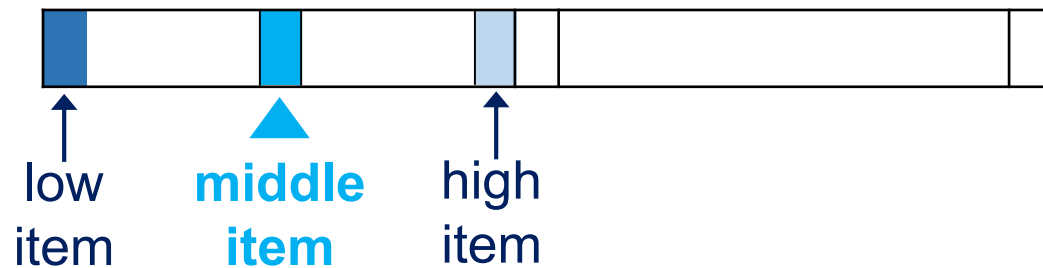
Binary Search Illustration



Is the middle item what we are looking for?

If not, is it lower or greater than the target item?

(Assume lower)



Binary Search steps: 0

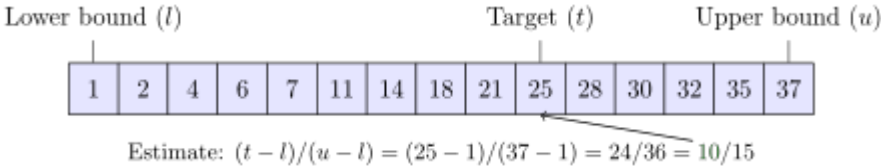
The array contains 20 elements, indexed 0 to 19. The elements are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19. A red circle with the number 13 and a red arrow points to the first element (index 0).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Value	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	

Algorithm	Best Time Complexity	Average Time Complexity	Worst Time Complexity	Worst Space Complexity
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

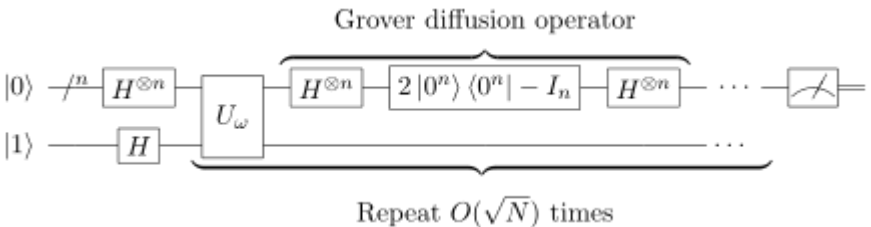
Other Searching Algorithm

Interpolation Search



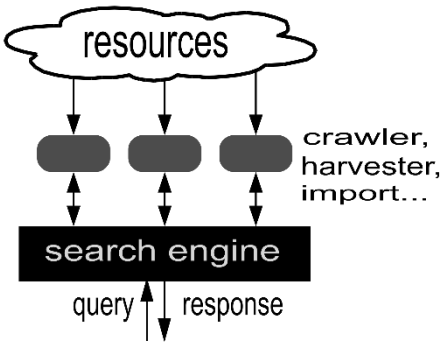
more like what people really do

Grover's Algorithm

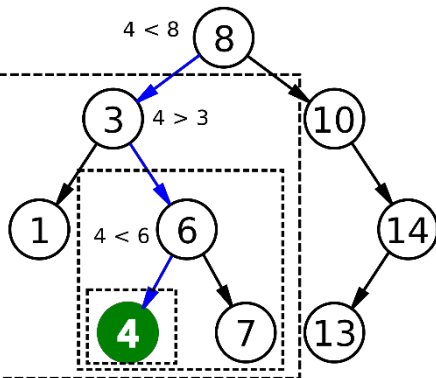


waiting for quantum computers to be built

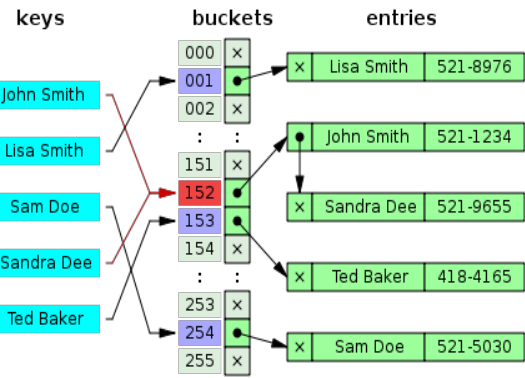
Indexed Searching



Binary Search Trees

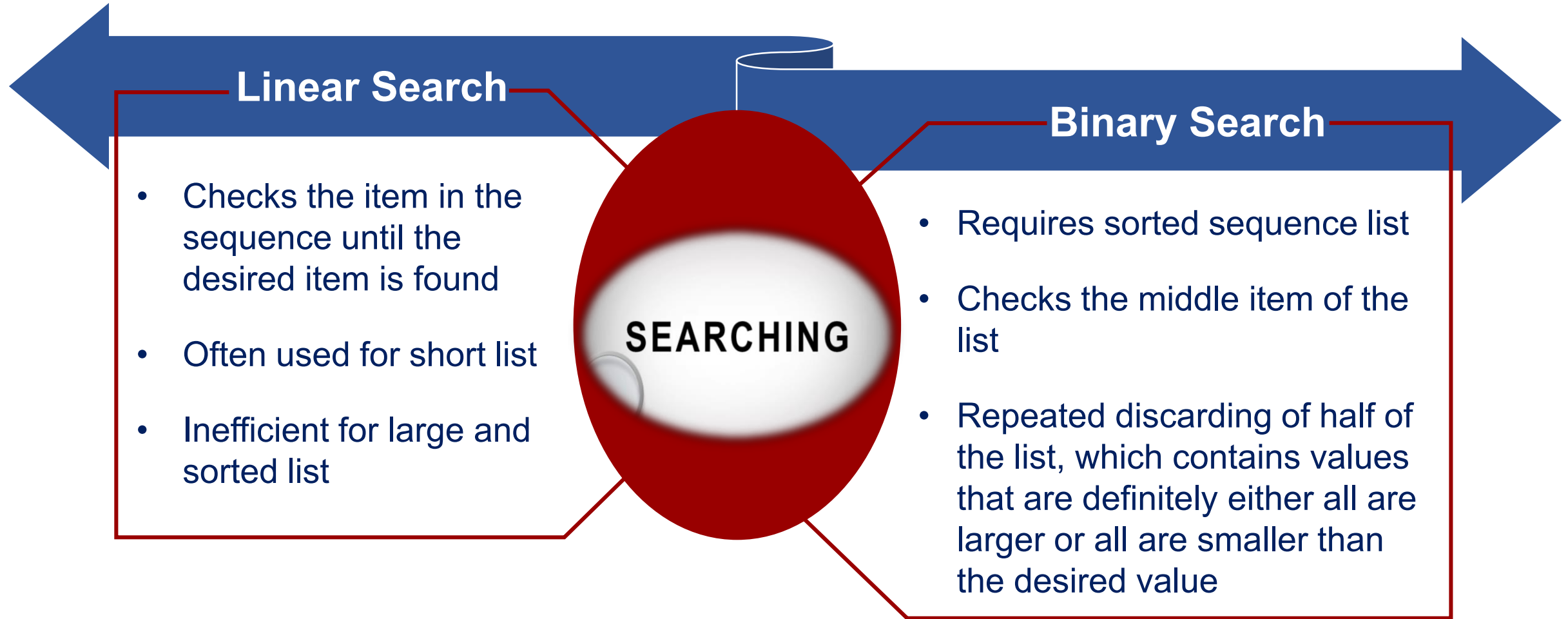


Hash Table Searching

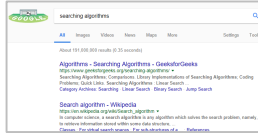


Best-first

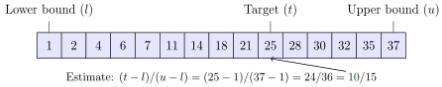
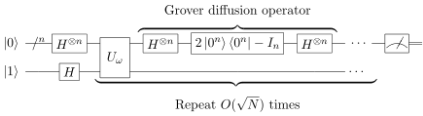
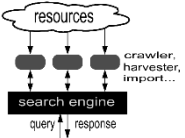
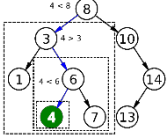
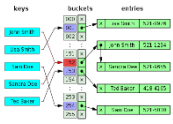




References for Images

No.	Slide No.	Image	Reference
1	3		Retrieved July 16, 2018 from https://www.google.com/ .
2	7		By Source, Fair use, retrieved July 16, 2018 from https://en.wikipedia.org/w/index.php?curid=22312809 .
3	7, 8, 9		Magnifying Glass [Online Image]. Retrieved July 16, 2018 from http://www.publicdomainfiles.com/show_file.php?id=13534684215801 .
4	8, 9		By Burger King - https://bk.com , Public Domain, retrieved July 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=53835318 .
5	10		Magnifier [Online Image]. Retrieved July 16, 2018 from https://pixabay.com/en/copyright-magnifier-magnifying-glass-389901/ .

References for Images

No.	Slide No.	Image	Reference
6	14		By Esquivallence - Own work, CC0, retrieved July 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=63614695 .
7	14		By Bender2k14 - Created in LaTeX code using Q-circuit. Source code follows this template., CC BY-SA 3.0, retrieved July 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=13524800 .
8	14		By Jakob Voss - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=3112366 .
9	14		By Binary_search_tree.svg: Booyabazookaderivative work: movax - Binary_search_tree.svg, Public Domain, retrieved July 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=11079967 .
10	14		By Jorge Stolfi - Own work, CC BY-SA 3.0, retrieved July 16, 2018 from https://commons.wikimedia.org/w/index.php?curid=6471915 .

No.	Slide No.	Image	Reference
11	14		Morville, P. (2010). 4-10 Best First with Google [Online Image]. Retrieved July 16, 2018 from https://www.flickr.com/photos/morville/4274337684/ .



Algorithm Design: Sorting

google.com/maps/search/coffee+shops/@1.3395072,103.6965417,15z

coffee shops

Rating at least
Any rating

Best Coffee Pte Ltd
3.4 ★★★★★ (16)
Coffee Shop · 959 Jurong West Street 92
Open until 12:00 AM

Eating House 815
4.0 ★★★★★ (172)
Coffee Shop · 815 Jurong West Street 81
Open until 12:00 AM

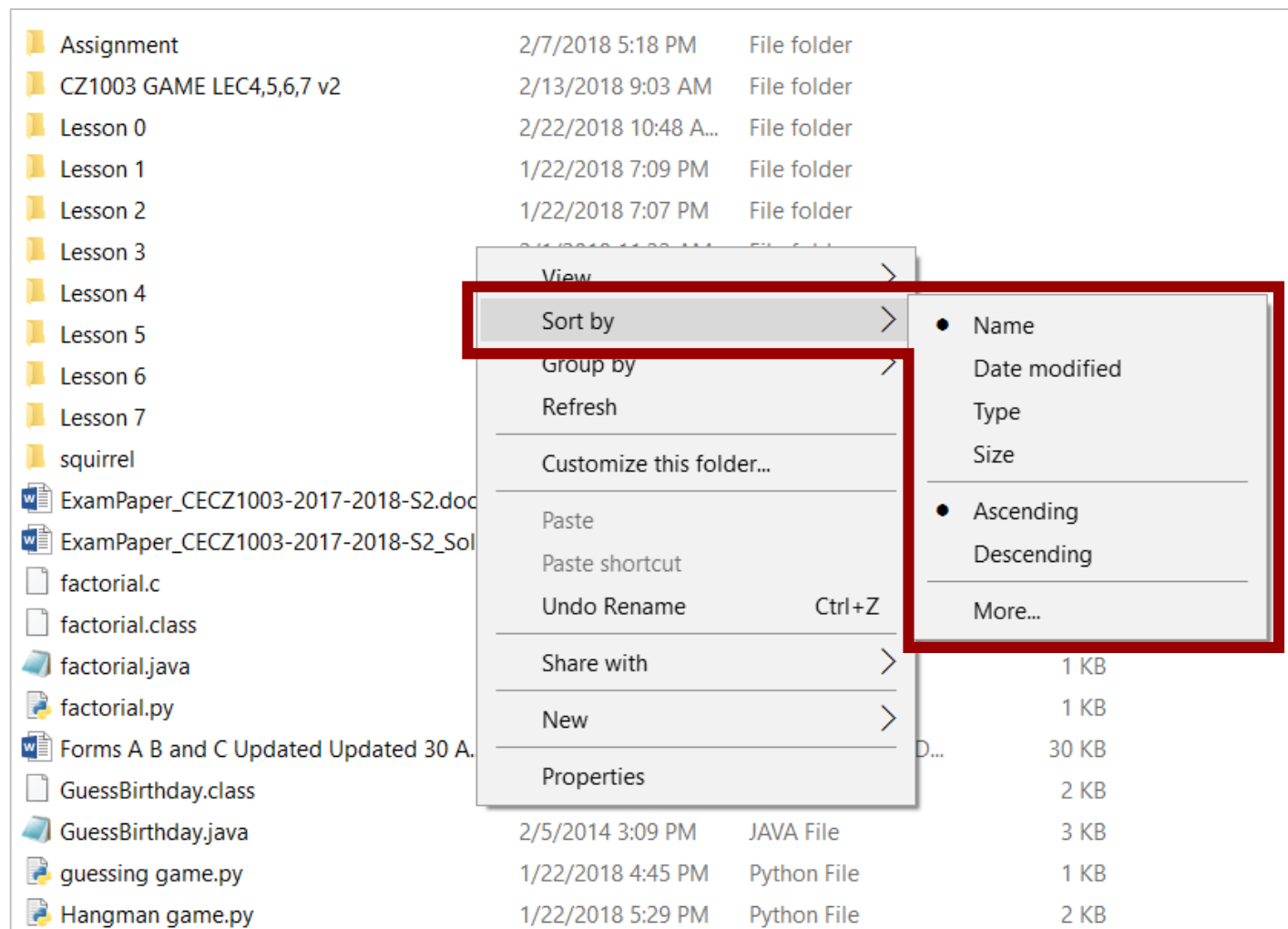
Block 651 Coffeeshop
4.1 ★★★★★ (41)
Coffee Shop · 651 Jurong West Street 61

You & Lai Coffee Foodlink
5.0 ★★★★★ (1)
Coffee Shop · 907 Jurong West Street 91

851 Coffeeshop
3.7 ★★★★★ (49)
Coffee Shop · 851 Jurong West Street 81
Open until 11:00 PM

Ya Kun Kaya Toast
4.0 ★★★★★ (55)
Coffee Shop · 1 Jurong West Central 2 #02-

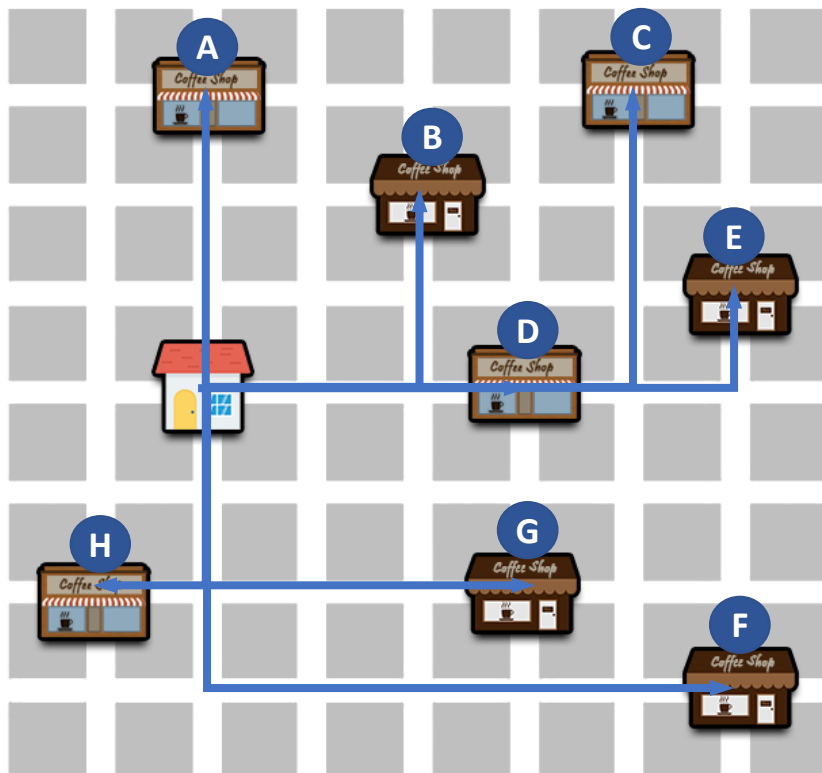
Map showing coffee shops and landmarks in the Jurong West area, including Nanyang Technological University, SAFRA Jurong, and Jurong ActiveSG Stadium.



Name	Date modified	Type	Size
Assignment	2/7/2018 5:18 PM	File folder	
CZ1003 GAME LEC4,5,6,7 v2	2/13/2018 9:03 AM	File folder	
Lesson 0	2/22/2018 10:48 A...	File folder	
Lesson 1	1/22/2018 7:09 PM	File folder	
Lesson 2	1/22/2018 7:07 PM	File folder	
Lesson 3			
Lesson 4			
Lesson 5			
Lesson 6			
Lesson 7			
squirrel			
ExamPaper_CECZ1003-2017-2018-S2.doc			
ExamPaper_CECZ1003-2017-2018-S2_Sol			
factorial.c			
factorial.class			
factorial.java			1 KB
factorial.py			1 KB
Forms A B and C Updated Updated 30 A...			30 KB
GuessBirthday.class			2 KB
GuessBirthday.java	2/5/2014 3:09 PM	JAVA File	3 KB
guessing game.py	1/22/2018 4:45 PM	Python File	1 KB
Hangman game.py	1/22/2018 5:29 PM	Python File	2 KB

Recall: Application of Looping

Find the Distance to N Locations



Coffeeshop	Distance
Coffeeshop A	3
Coffeeshop B	4
Coffeeshop C	7
Coffeeshop D	3
Coffeeshop E	6
Coffeeshop F	8
Coffeeshop G	5
Coffeeshop H	3

**Sort by
distance**



Coffeeshop	Distance
Coffeeshop A	3
Coffeeshop D	3
Coffeeshop H	3
Coffeeshop B	4
Coffeeshop G	5
Coffeeshop E	6
Coffeeshop C	7
Coffeeshop F	8



At the end of this lesson, you should be able to:

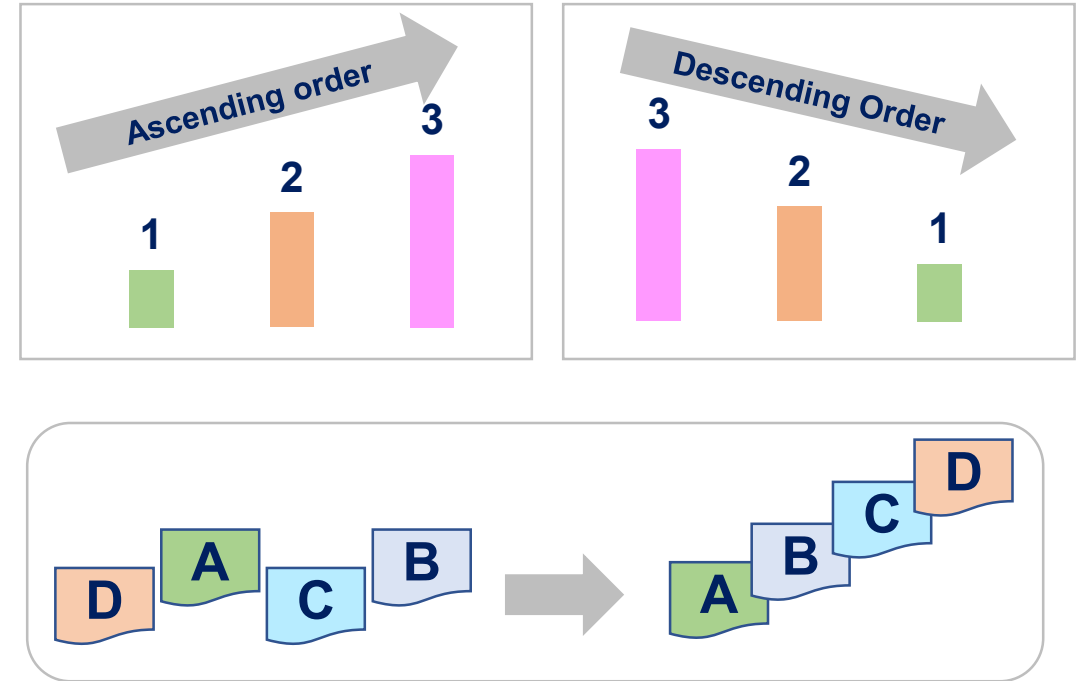
- Describe the process of sorting
- Explain the importance of different types of sorting algorithms
- Perform sorting algorithms particularly sorting alphabetically or numerically
- Sort an array using bubble sort and merge sort
- Apply your knowledge and understanding of sorting algorithms to your problem solving
- Recognize that “no single” best sorting applies to all scenarios

Topic Outline



Sorting Algorithm

- In computer science, it is an algorithm that puts elements in a list of a certain order.
- The most frequently used orders are **numerical** and **alphabetical orders**.
- Efficient sorting is important for optimizing the efficiency of other algorithms (such as search and merge algorithms).



Sorting Algorithms: Review of Complexity

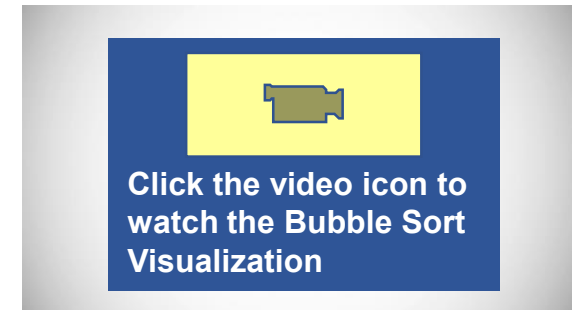
Name	Best	Average	Worst	Memory	Stable	Method	Other Noted
Timesort	-	$n \log n$	$n \log n$	n	Yes	Insertion & Merging	n comparisons when the data is already sorted.
Bubble sort	n	n^2	n^2	1	Yes	Exchanging	Tiny code
Cocktail sort	-	-	n^2	1	Yes	Exchanging	
Comb sort	-	-	-	1	No	Exchanging	Small code size
Gnome sort	-	-	n^2	1	Yes	Exchanging	Tiny code size
Selection sort	n^2	n^2	n^2	1	No	Selection	Its stability depends on the implementation. Used to sort this table in safari or other Webkit web browser [2]
Insertion sort	n	n^2	n^2	1	Yes	Insertion	Average case is also $o(n + d)$, where d is the number of inversions
Cycle Sort	-	n^2	n^2	1	No	Insertion	In-place with theoretically optimal number of writes
Shell Sort	-	-	$n \log^2 n$	1	No	Insertion	
Binary tree sort	-	$n \log n$	$n \log n$	n	Yes	Insertion	When using a self-balancing binary search tree
Library sort	-	$n \log n$	n^2	n	Yes	Insertion	
Merge sort	$n \log n$	$n \log n$	$n \log n$	Depends	Yes	Merging	Used to sort this table in Firefox [3]
In-place merge sort	-	$n \log n$	$n \log n$	1	Depends	Merging	Example implementation here: [4] ; can be implemented as a stable sort based on stable in-place merging: [5]
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Smoothsort	-	-	$n \log n$	1	No	Selection	An adaptive sort – n comparisons when the data is already sorted, and $o(0)$ swaps.
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	Depends	Partitioning	Can be implemented as a stable sort depending on how the pivot is handled. Naïve variants use $o(n)$ space
Introsort	-	$n \log n$	$n \log n$	$\log n$	No	Hybrid	Used in SGI STL Implementations
Patience sorting	-	-	$n \log n$	n	No	Insertion & Selection	Finds all the longest increasing subsequences within $O(n \log n)$
Strand sort	-	$n \log n$	n^2	n	Yes	Selection	
Tournament sort	-	$n \log n$	$n \log n$			Selection	

- Most of the primary sorting algorithms run on different space and time complexity.
- Time Complexity is defined to be the time the computer takes to run a program (or algorithm in our case).
- Space complexity is defined to be the amount of memory the computer needs to run a program.

 *More on this later..*

Bubble Sort

- Sometimes referred to as “sinking sort”.
- One of the simplest sorting algorithm.
- Repeatedly steps through the list to be sorted, compares each pair of adjacent items, and swaps them if they are in the wrong order.
- The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.
- The algorithm, which is a comparison sort, is named for the way smaller or larger elements "bubble" to the top of the list.

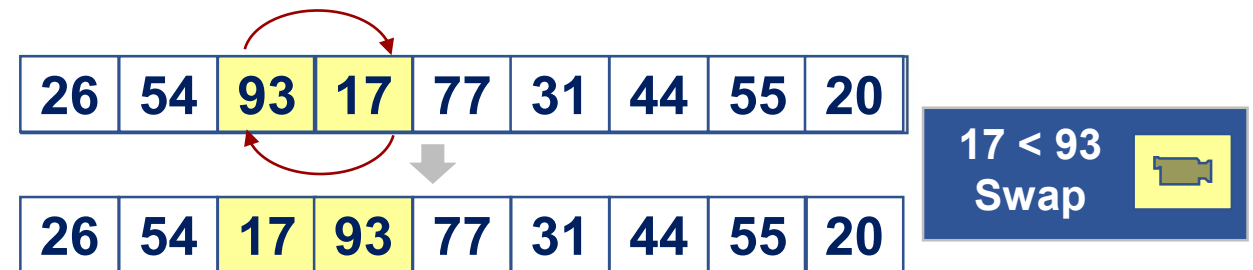
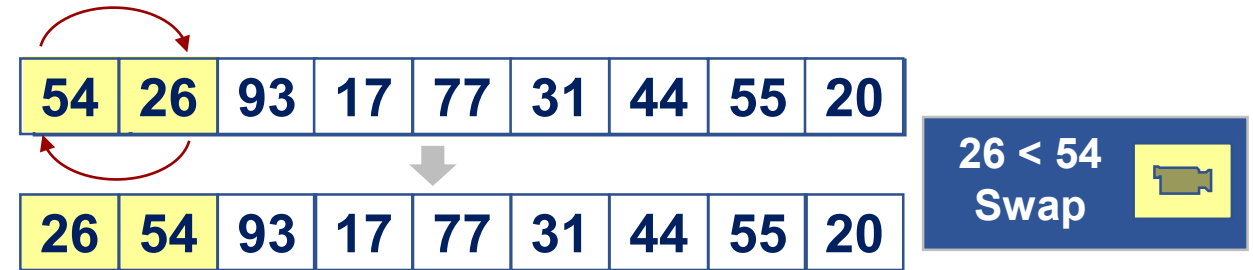


Bubble Sort: Step by Step

The bubble sort makes multiple passes through a list.

For each pass, the bubble sort is operated as follows:

- 1 Compare the first two number, and if the second number is smaller than the first, then the numbers are swapped.
- 2 Then move down one number, compare the number and the number that follows it, and swap the two numbers, if necessary.
- 3 Repeat this process.



Bubble Sort: First Pass

First Pass



until the last two numbers of the array have been compared

- Largest number has now moved to the bottom of the list.
- Each sequence of comparison is called a pass.

54	26	93	17	77	31	44	55	20	Swap
26	54	93	17	77	31	44	55	20	No Swap
26	54	93	17	77	31	44	55	20	Swap
26	54	17	93	77	31	44	55	20	Swap
26	54	17	77	93	31	44	55	20	Swap
26	54	17	77	31	93	44	55	20	Swap
26	54	17	77	31	44	93	55	20	Swap
26	54	17	77	31	44	55	93	20	Swap
26	54	17	77	31	44	55	20	93	93 in place after the first pass

First Pass

Bubble Sort: Second Pass

At the **start of the second pass**,

- the largest value is now in place
- there are $n - 1$ items left to sort
- meaning, there will be $n - 2$ pairs



**93 in place after
the first pass**



Second Pass



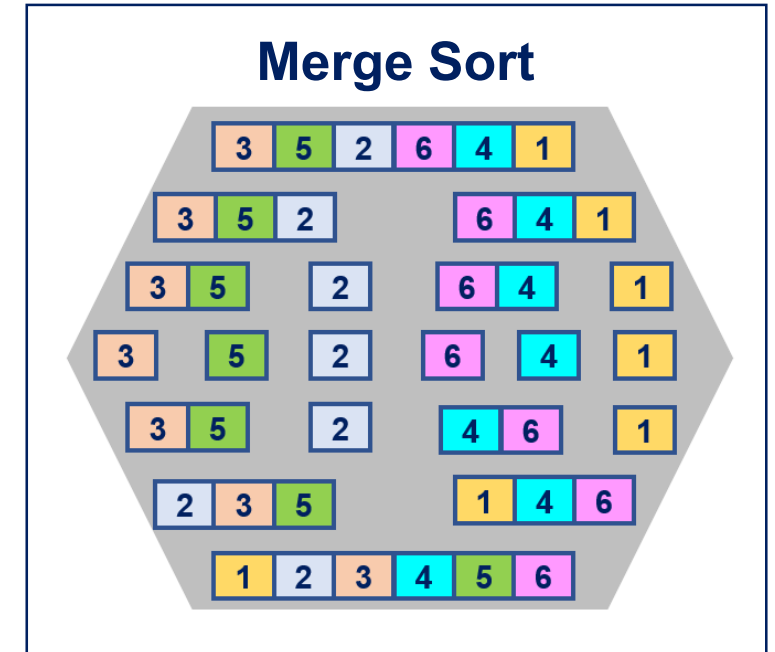
Bubble Sort: Repeat Process ($n - 1$ passes) to End

- Since each pass places the next largest value in place, the total number of passes necessary will be $n - 1$.
- After completing the $n - 1$ passes, the smallest item must be in the correct position with no further processing required.

17	20	26	31	44	54	55	77	93
----	----	----	----	----	----	----	----	----

Merge Sort

- Merge sort is an example of a divide-and-conquer style of algorithm.
- A problem is repeatedly broken up into sub-problems, often using recursion, until they are small enough to be solved.
- The solutions are combined to solve the larger problem.
- The idea behind merge sort is to break the data into parts that can be sorted trivially, then combine those parts knowing that they are sorted.



Click the video icon to
watch the Merge Sort
Visualization

Merge Adjacent List

This means we have a list on the left and a list on the right.

Step 1

Compare the first elements of both lists one by one.

Step 2

Move the smaller element out of the list that it was found in. Add this value to the list of “sorted items”.

Step 3

Repeat the process until only a single list remains.

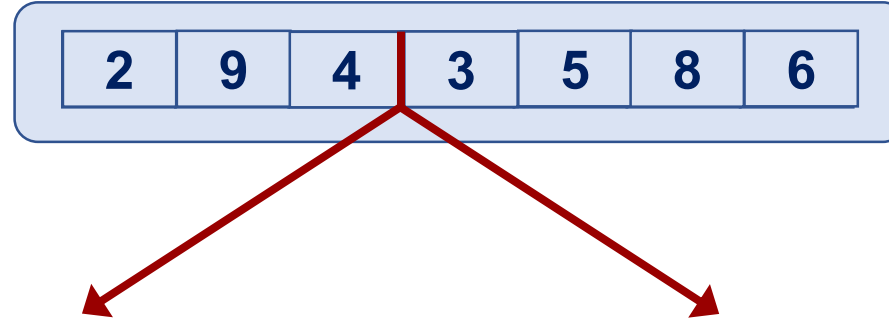
Step 4

One list should still contain elements. This list is sorted. Move its contents into the result list.



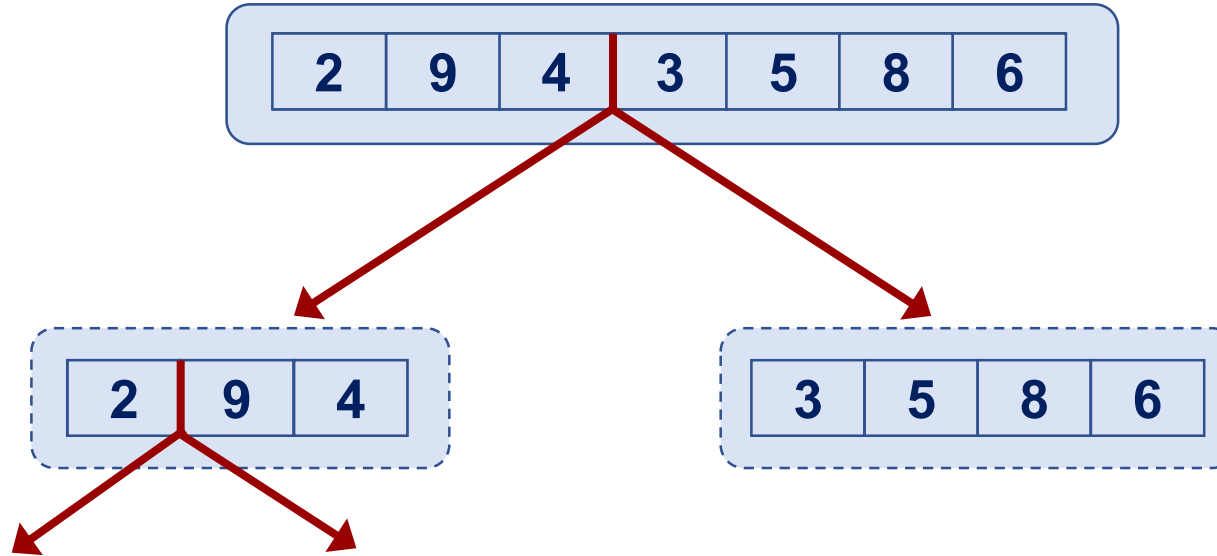
Merge Sort: Execution Example

Partition



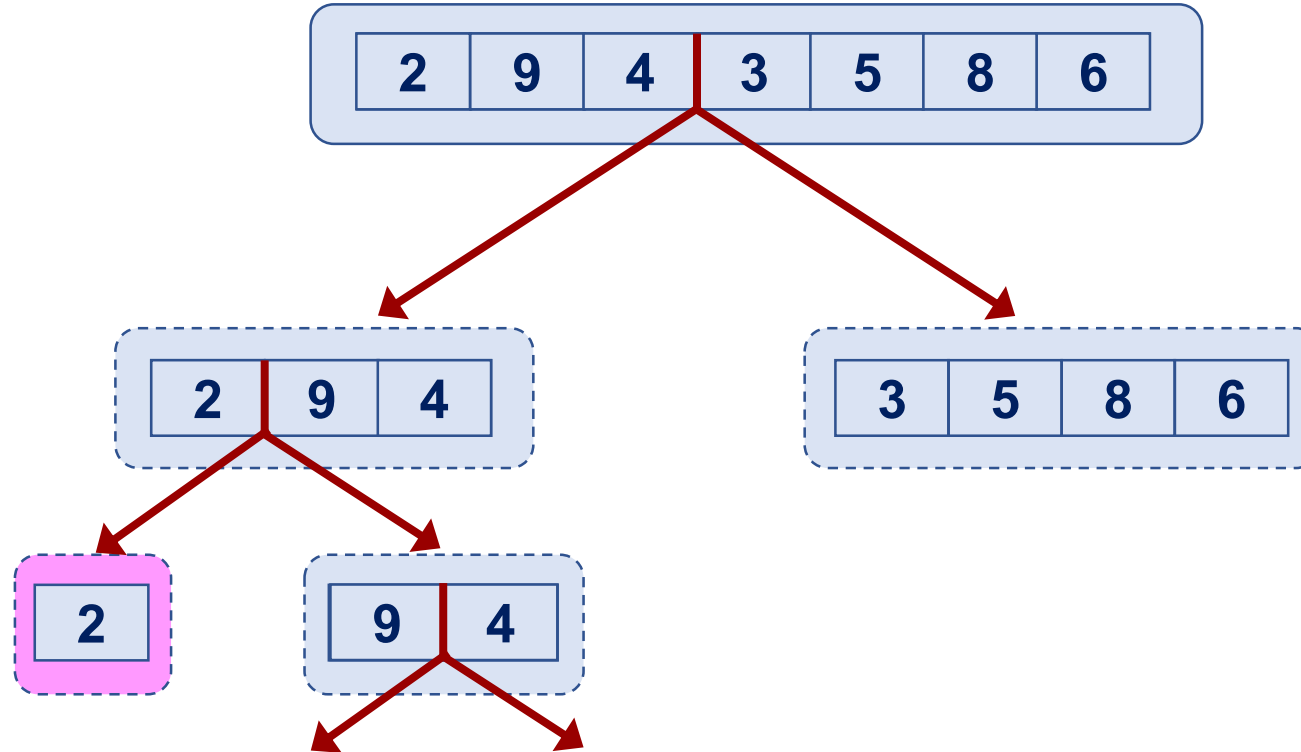
Merge Sort: Execution Example (Cont'd)

Partition



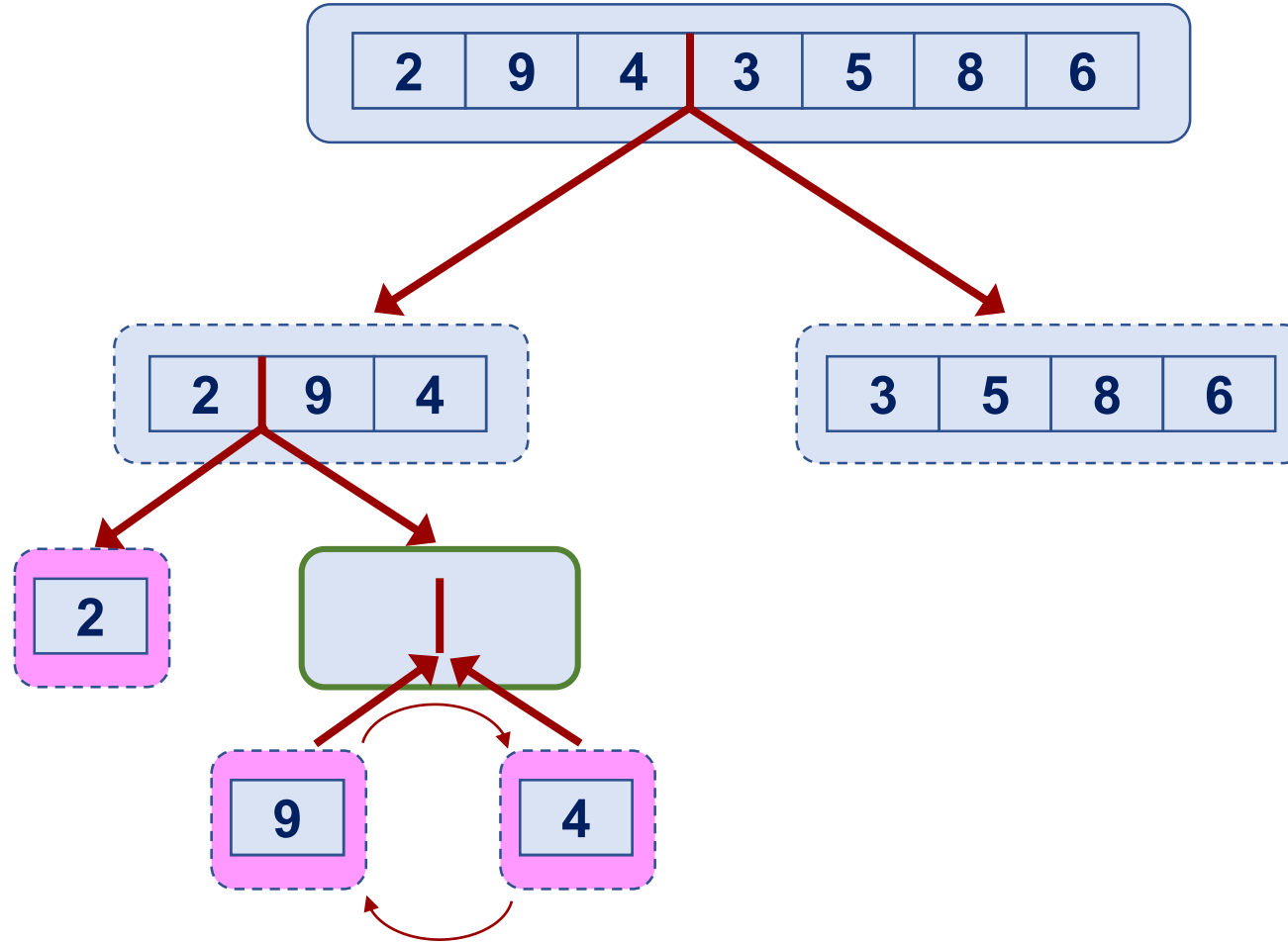
Merge Sort: Execution Example (Cont'd)

Partition



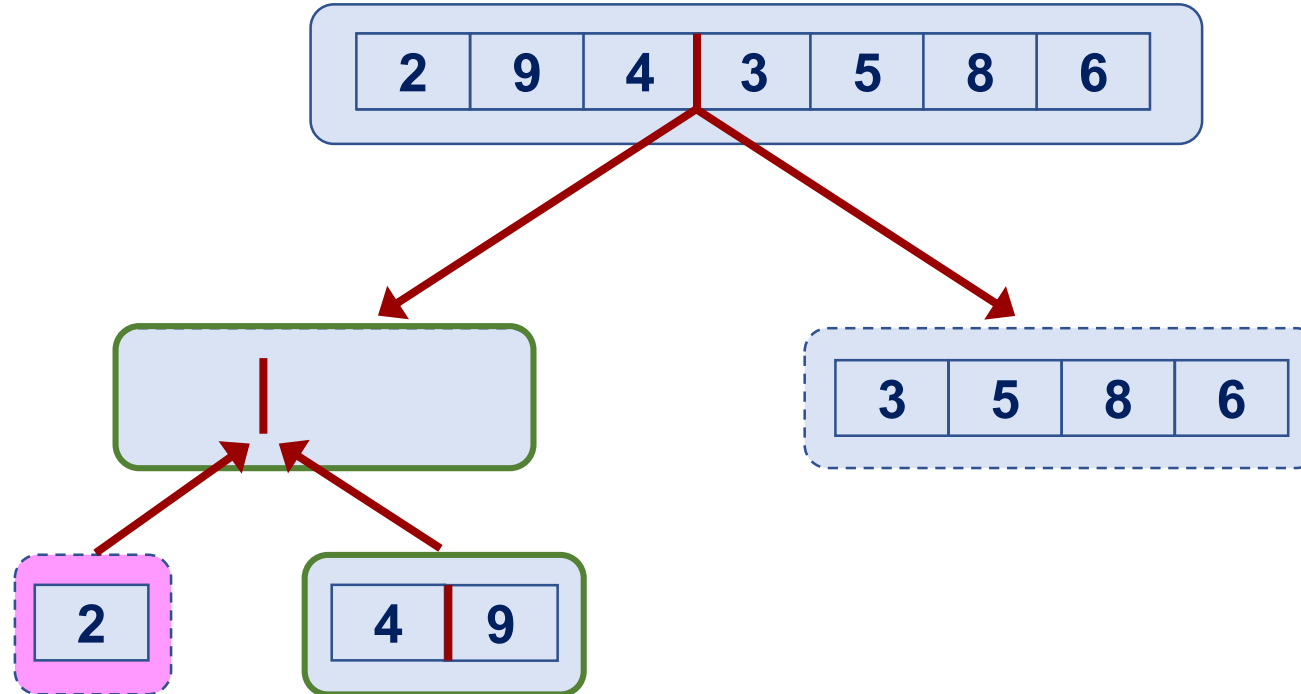
Merge Sort: Execution Example (Cont'd)

Merge

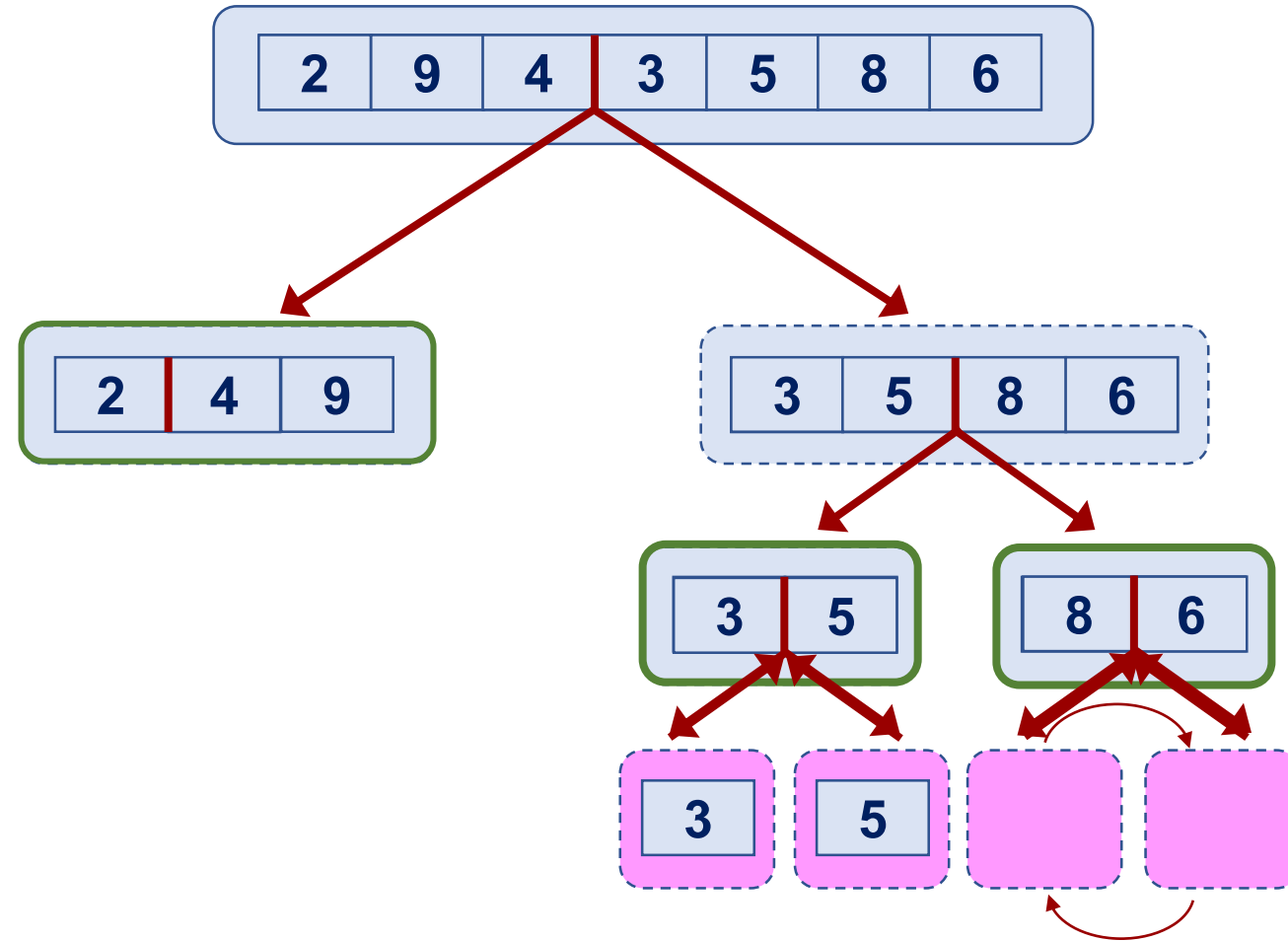


Merge Sort: Execution Example (Cont'd)

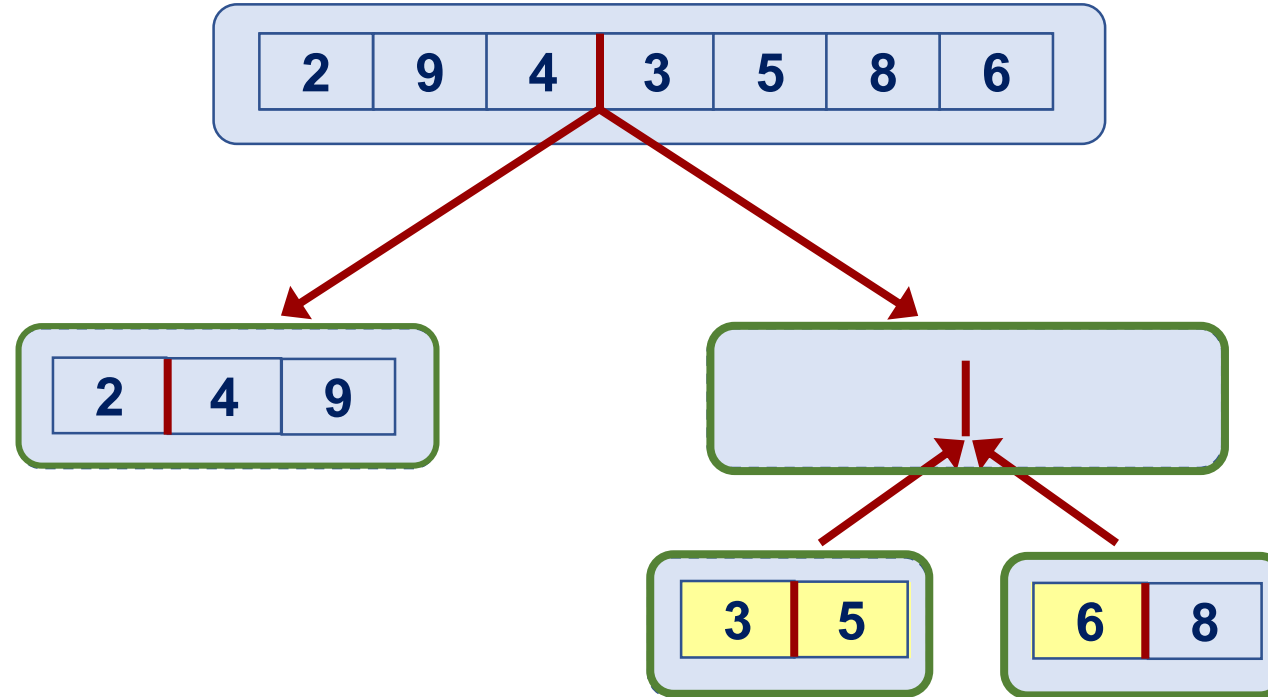
Merge



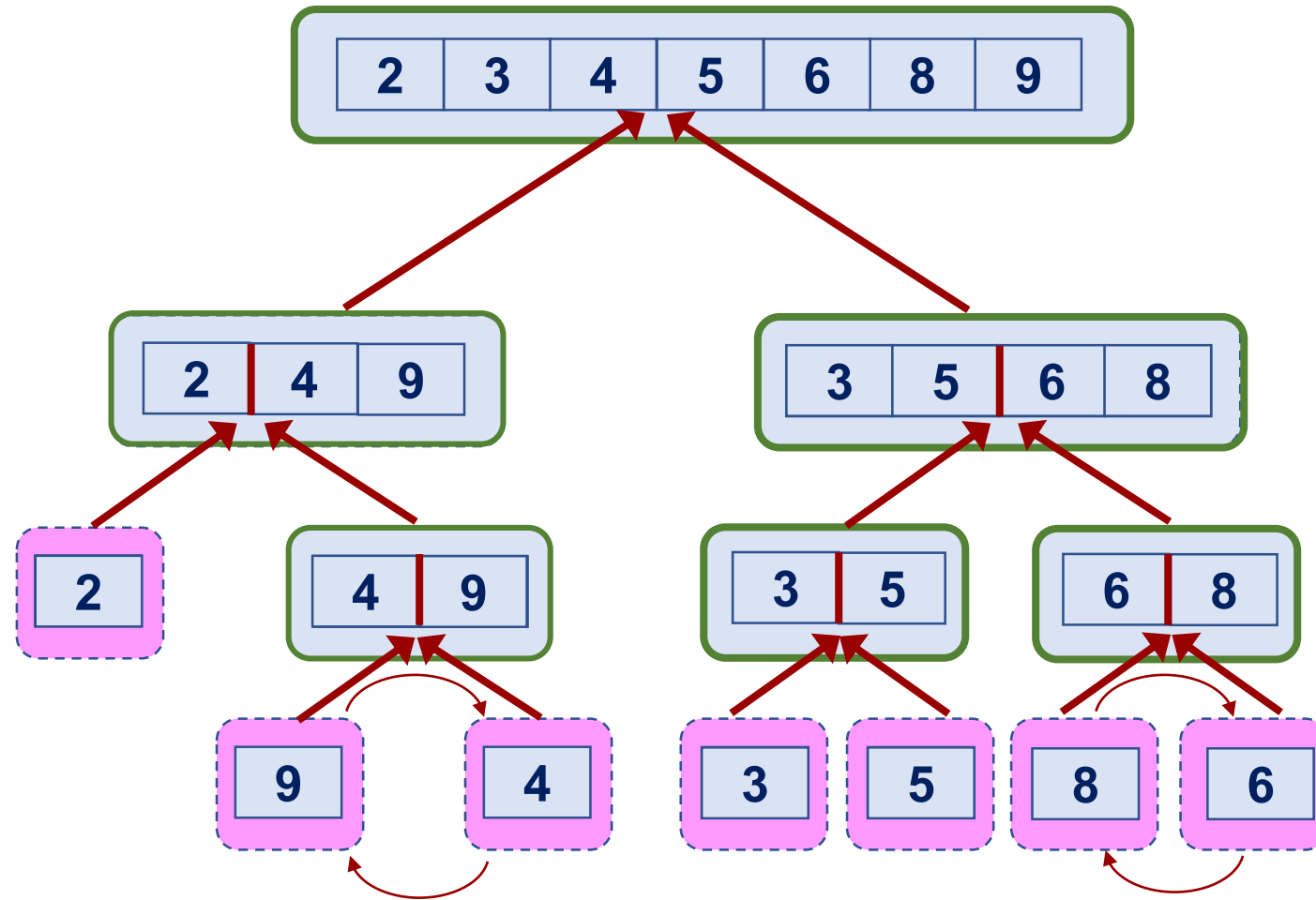
Merge Sort: Execution Example (Cont'd)

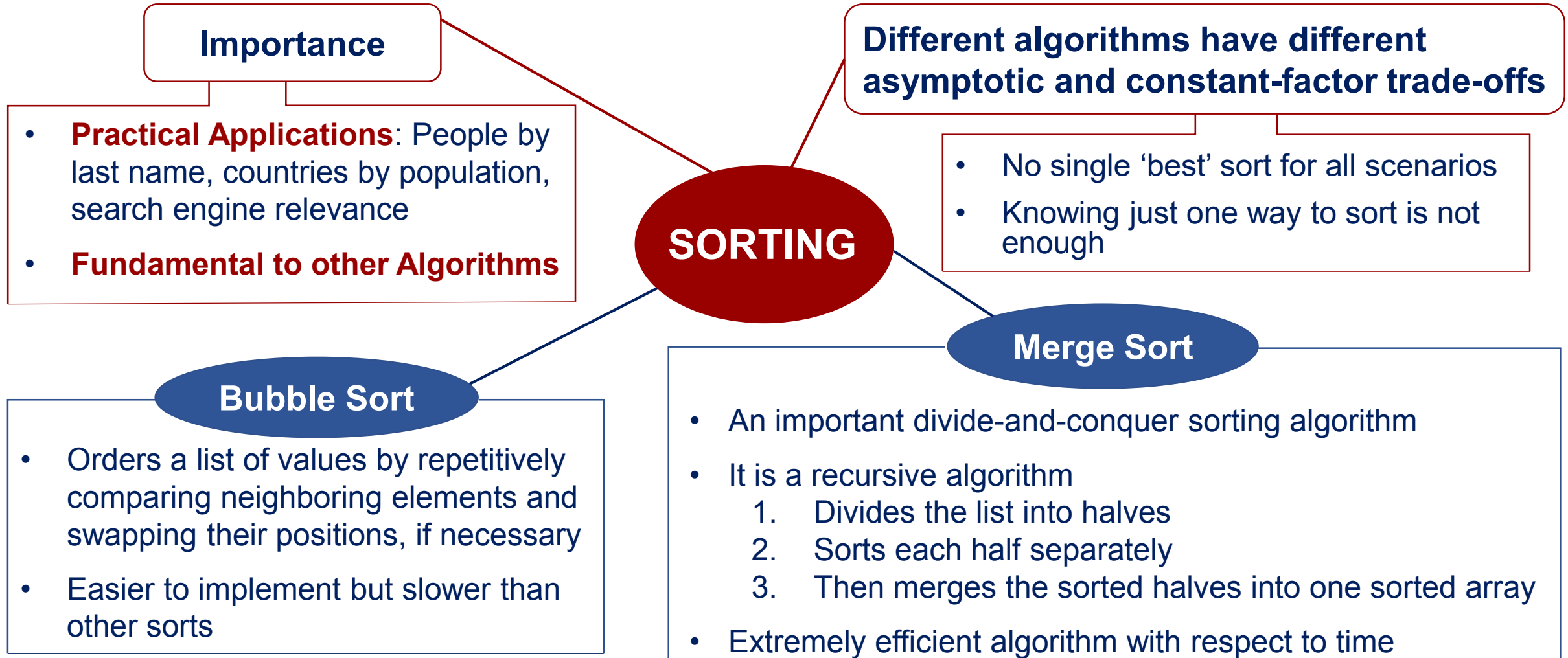


Merge Sort: Execution Example (Cont'd)

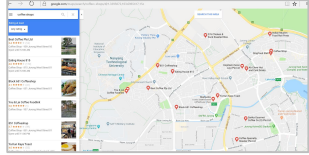


Merge Sort: Execution Example (Cont'd)





References for Images

No.	Slide No.	Image	Reference
1	3		Google Maps
2	9		By User:Bobarino - Made by following Information.png, CC BY-SA 3.0, retrieved April 18, 2018 from https://en.wikipedia.org/w/index.php?curid=9180601 .
3	10		Bubble Soap [Online Image]. Retrieved July 3, 2018 from https://www.maxpixel.net/Colorful-Fly-Soap-Bubbles-Make-Soap-Bubbles-2405969 .



Algorithm Implementation: Searching in Python



At the end of this lesson, you should be able to:

- Use index method in Python
- Explain the importance of coding searching algorithms in Python
- Code linear search in Python
- Code binary search in Python
- Identify other search algorithms written in Python
- Apply your knowledge and understanding of searching algorithms to your problem solving in Python

Topic Outline



Linear Search: Recall

- Iterates over the sequence, one item at a time, until the specific item is found or all items have been examined
 - The element that needs to be found is called a search key
- Linear search/ sequential search
 - Intuitive approach
 - Starts at the first item
 - Is it the one I am looking for?
 - If not, goes to next item
 - Repeats until found or all the items are checked
- This approach is necessary if items are not sorted

  **in North Spine Plaza?**

List of Food & Beverage in **North Spine**



Bakery Cuisine

Subway

Peach Garden Chinese Restaurant

Mr Bean

Pizza Hut

FOUND

The Soup Spoon Union

North Spine Food Court

Linear Search in Python

```
foodList = ['Bakery Cuisine', 'Subway', 'Peach Garden Chinese Restaurant',  
            'Mr Bean', 'Pizza Hut', 'The Soup Spoon Union',  
            'North Spine Food Court']  
  
for item in foodList:  
    if item == 'Pizza Hut':  
        print('Pizza Hut is in list')  
        break
```



Python List `index()` Method

Description

The method `index()` returns the lowest index in list that *obj* appears.

The `index` method does a linear search and stops at the first matching item.

If no matching item is found, it raises a `ValueError` exception.

Python List `index()` Method: Example

```

0      1      2
foodList = ['Bakery Cuisine', 'Subway', 'Peach Garden Chinese Restaurant',
            3      4
            'Mr Bean', 'Pizza Hut', 'The Soup Spoon Union',
            'North Spine Food Court']

```

```

print ("Index for Pizza Hut : ", foodList.index( 'Pizza Hut' ))
print ("Index for Pizza Hut : ", foodList.index( 'pizza hut' ))

```



Index for Pizza Hut : 4

← This method returns index of the found object.

Traceback (most recent call last):

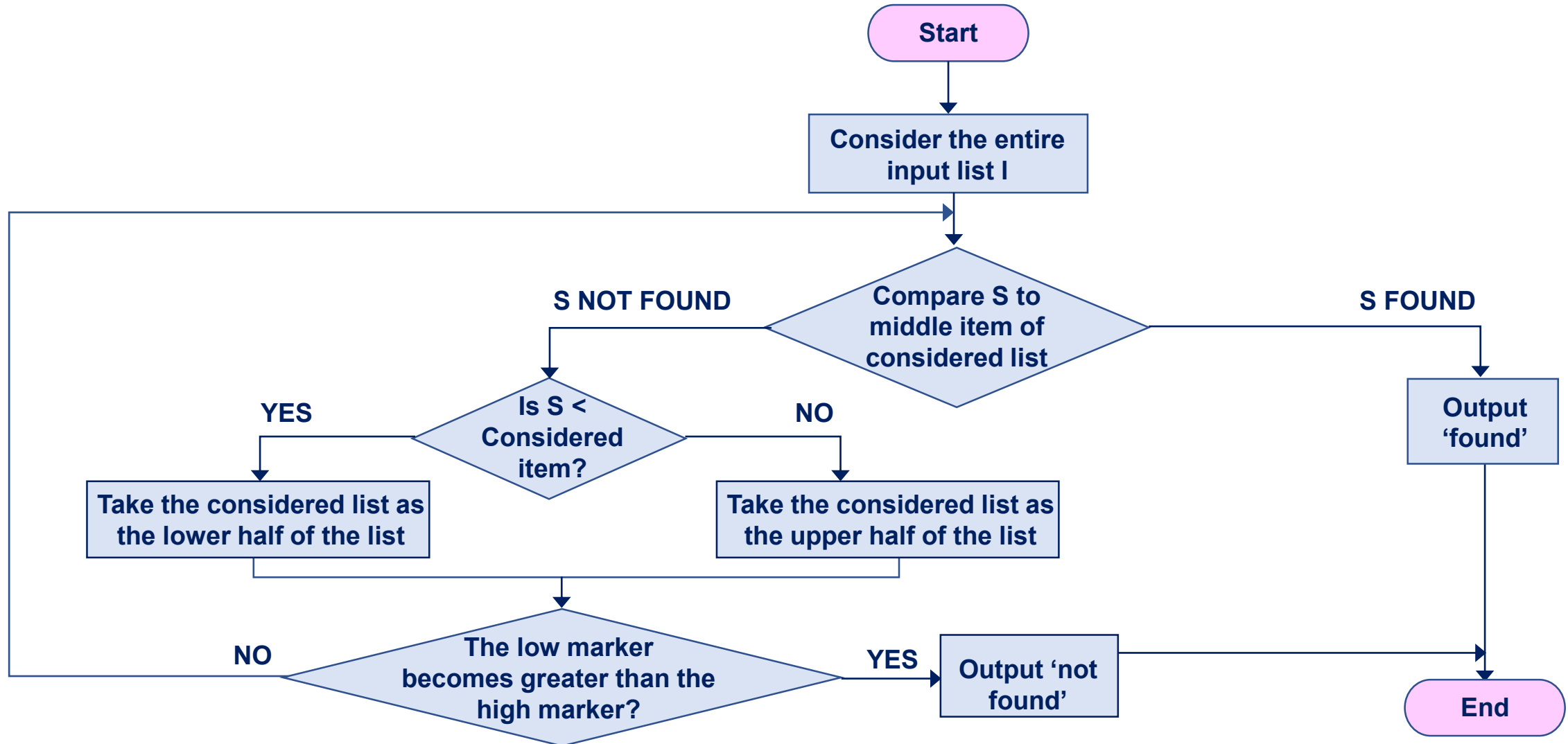
File "C:/New CX1003/7B2/indexPizzahut.py", line 6, in <module>

print ("Index for Pizza Hut : ", foodList.index('pizza hut'))

ValueError: 'pizza hut' is not in list

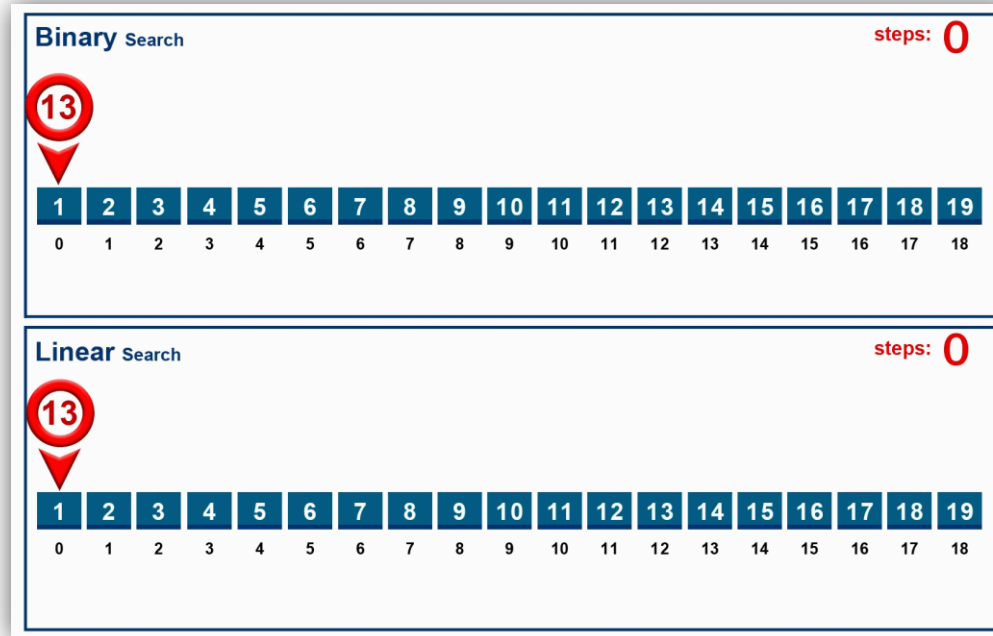
← Otherwise, it raises an exception indicating that the value is not found.

Binary Search Flowchart



Linear Search vs. Binary Search

Variance of Hi-Low
Number Guessing Game



Algorithm	Best Time Complexity	Average Time Complexity	Worst Time Complexity	Worst Space Complexity
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Iterative Binary Search

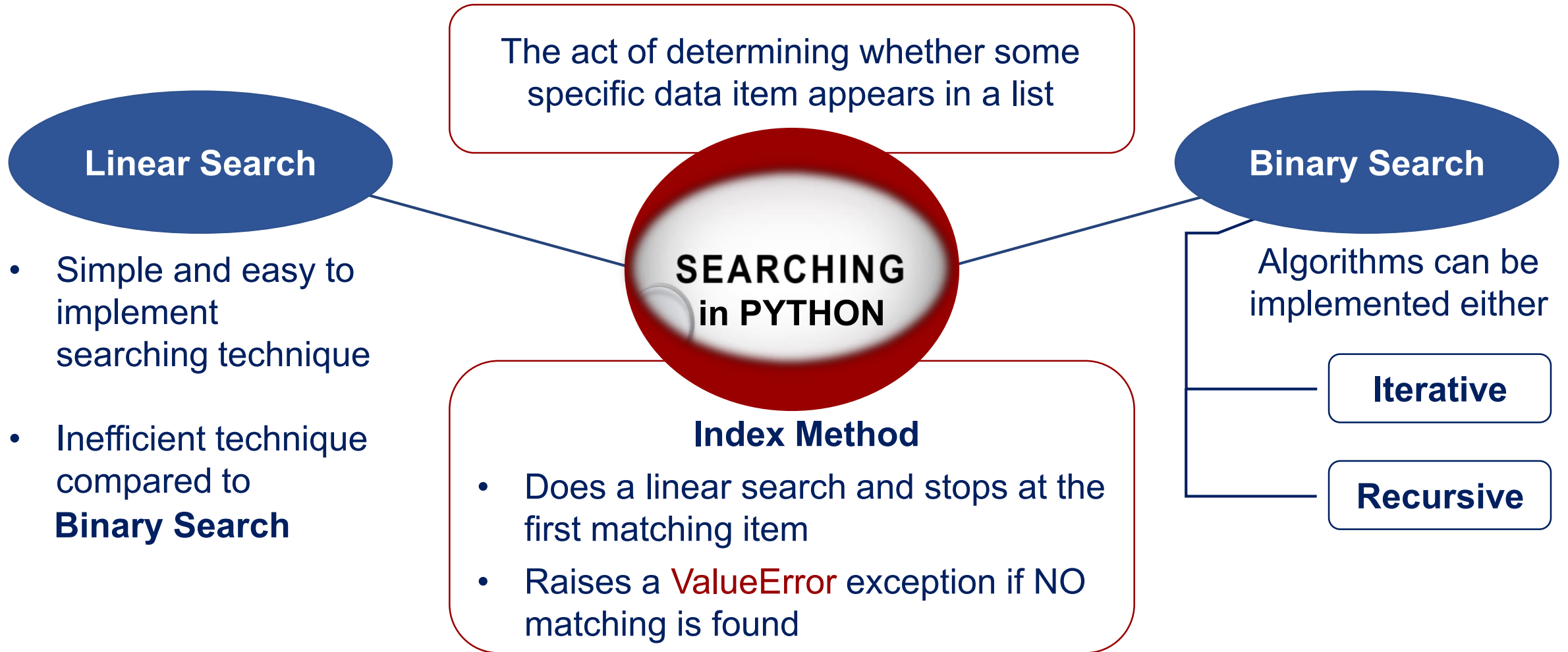
```
def binarySearch( items, target ) :  
    # Start with the entire list  
    low = 0  
    high = len(items) - 1  
    # Repeatedly subdivide the list in half until the target is found  
    while low <= high :  
        # locates the middle item of the list  
        mid = (low + high) // 2  
        # compares middle item with the search key  
        if items[mid] == target:  
            return True  
        # target is less than middle item?  
        elif target < items[mid] :  
            high = mid - 1  
        # target is greater than middle item?  
        else :  
            low = mid + 1  
    return False  
  
numbers = range(1, 20, 1)  
search_key = 7  
if (binarySearch( numbers, search_key )):  
    print (search_key, "is in the list")  
else:  
    print (search_key, "is not in the list")
```



Recursive Binary Search

```
def binary_search(items, target, low = 0, high = None):  
    if high == None:  
        high = len(items) - 1  
  
    if low > high:  
        return False  
  
    mid = (low + high) // 2  
  
    if target == items[mid]:  
        return True  
    elif target > items[mid]:  
        return binary_search(items, target, low = (mid + 1), high = high)  
    else:  
        return binary_search(items, target, low = low, high = (mid-1))  
  
numbers = range(1, 50, 1)  
search_key = 34  
if (binary_search( numbers, search_key, 0 )):  
    print (search_key, "is in the list")  
else:  
    print (search_key, "is not in the list")
```





References for Images

No.	Slide No.	Image	Reference
1	5		By Source, Fair use, retrieved July 16, 2018 from https://en.wikipedia.org/w/index.php?curid=22312809 .
2	5		Magnifying Glass [Online Image]. Retrieved July 16, 2018 from http://www.publicdomainfiles.com/show_file.php?id=13534684215801 .
3	All pages with Python codes		Python Logo [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/language-logo-python-2024210/ .



Algorithm Implementation: Sorting in Python



At the end of this lesson, you should be able to:

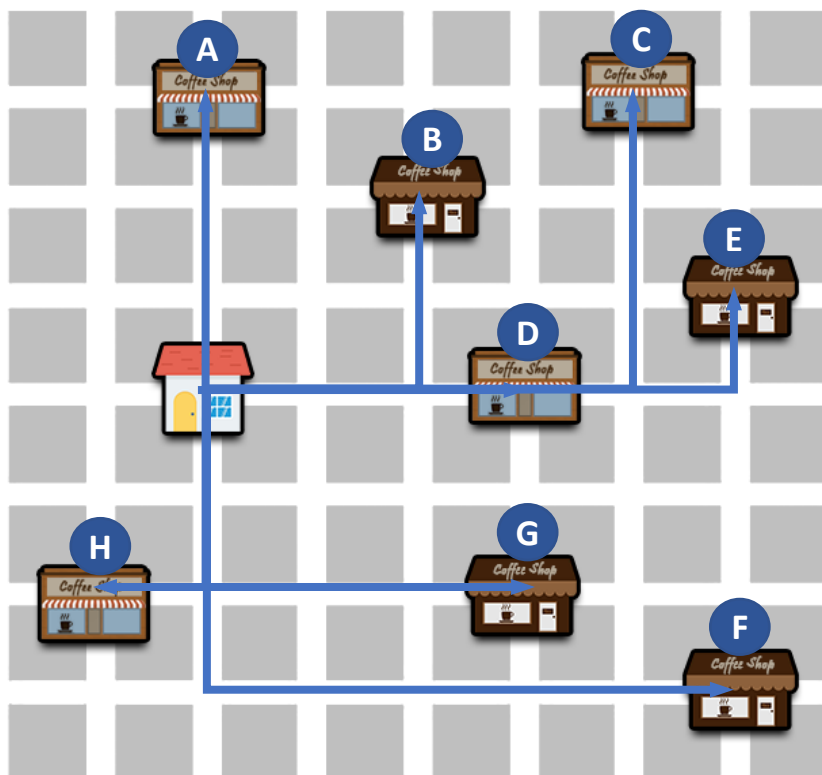
- Use sorted function in Python
- Explain the importance of coding sort algorithms in Python
- Code bubble sort in Python
- Code merge sort in Python
- Read and understand other sorting algorithms written in Python
- Apply your knowledge and understanding of sorting algorithms to your problem solving in Python

Topic Outline



Recall: Application of Looping

Find the Distance to N Locations



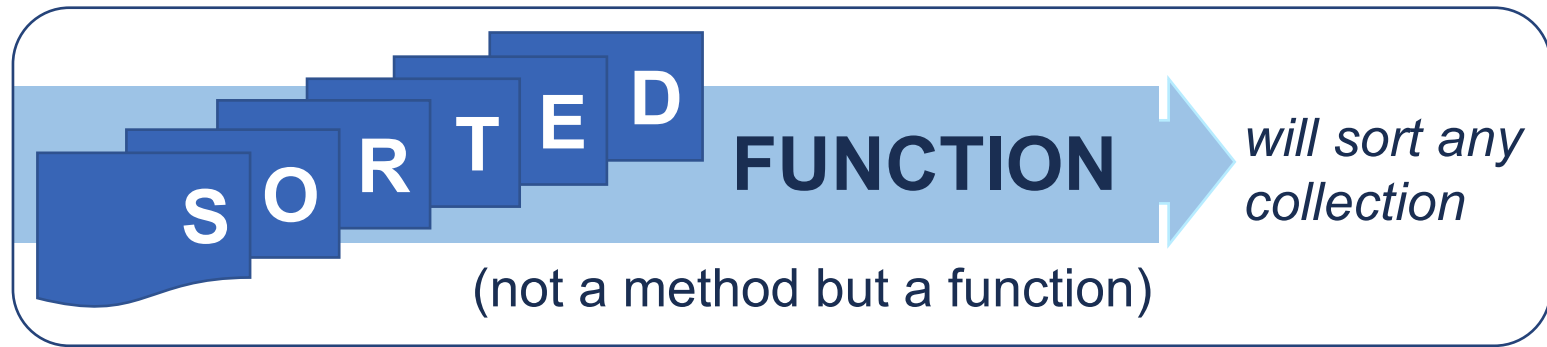
Coffeeshop	Distance
Coffeeshop A	3
Coffeeshop B	4
Coffeeshop C	7
Coffeeshop D	3
Coffeeshop E	6
Coffeeshop F	8
Coffeeshop G	5
Coffeeshop H	3

**Sort by
distance**



Coffeeshop	Distance
Coffeeshop A	3
Coffeeshop D	3
Coffeeshop H	3
Coffeeshop B	4
Coffeeshop G	5
Coffeeshop E	6
Coffeeshop C	7
Coffeeshop F	8

Sorted Function



Separates the collection into individual elements

- Sorts those elements

Returns the elements in a sorted list

- Returns a new sorted list without changing the original list

Sort by Distance

Python provides a module named `operator` that contains the useful `itemgetter` for sorting values other than the first and allows sorting on multiple values.



```
from operator import itemgetter
restaurant_info = [['Kentucky', 15, 6, 'Fried chicken'],
                   ['Macdonald', 12, 5, 'Burger'],
                   ['Subway', 13, 7, 'Sandwiches']]

# field 1 (index 0): name of restaurant
# field 2 (index 1): distance of restaurant
# field 3 (index 2): average price per person of the restaurant
# field 4 (index 3): signature dish of the restaurant

sort_info = sorted(restaurant_info, key = itemgetter(1))
print('sort by distance', sort_info)
```

Sort by Any Field

```
from operator import itemgetter
restaurant_info = [['Kentucky', 15, 6, 'Fried chicken'],
                   ['Macdonald', 12, 5, 'Burger'],
                   ['Subway', 13, 7, 'Sandwiches']]

# field 1 (index 0): name of restaurant
# field 2 (index 1): distance of restaurant
# field 3 (index 2): average price per person of the restaurant
# field 4 (index 3): signature dish of the restaurant
sort_field = ['name', 'distance', 'price', 'food']

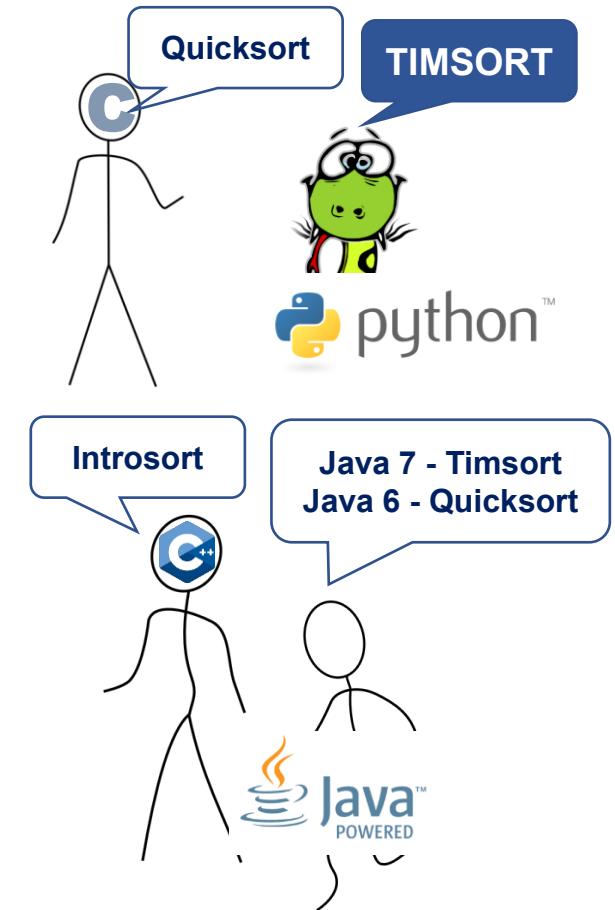
for i in range (len(sort_field)):
    sort_info = sorted (restaurant_info, key = itemgetter(i))
    print('sort by', sort_field[i], sort_info)
```



What Algorithm does Python's Sorted() Use?

TIMSORT Hybrid sorting algorithm used by Python

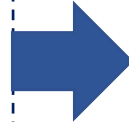
- Derived from merge sort and insertion sort and designed to perform well on many kinds of real-world data
- Invented by Tim Peters in 2002 for use in the Python programming language
- Finds subsets of the data that are already ordered, and uses the subsets to sort the data more efficiently. This is done by merging an identified subset, called a run, with existing runs until certain criteria are fulfilled
- Timsort has been Python's standard sorting algorithm since version 2.3
- Now, also used to sort arrays in Java SE 7 and on the Android platform



Importance of Basic Algorithm Coding



**Why is there a need
to learn how to code
basic sort algorithm?**



Training for the Future

Bubble Sort Algorithm

Step 1: Get the length of the sequence

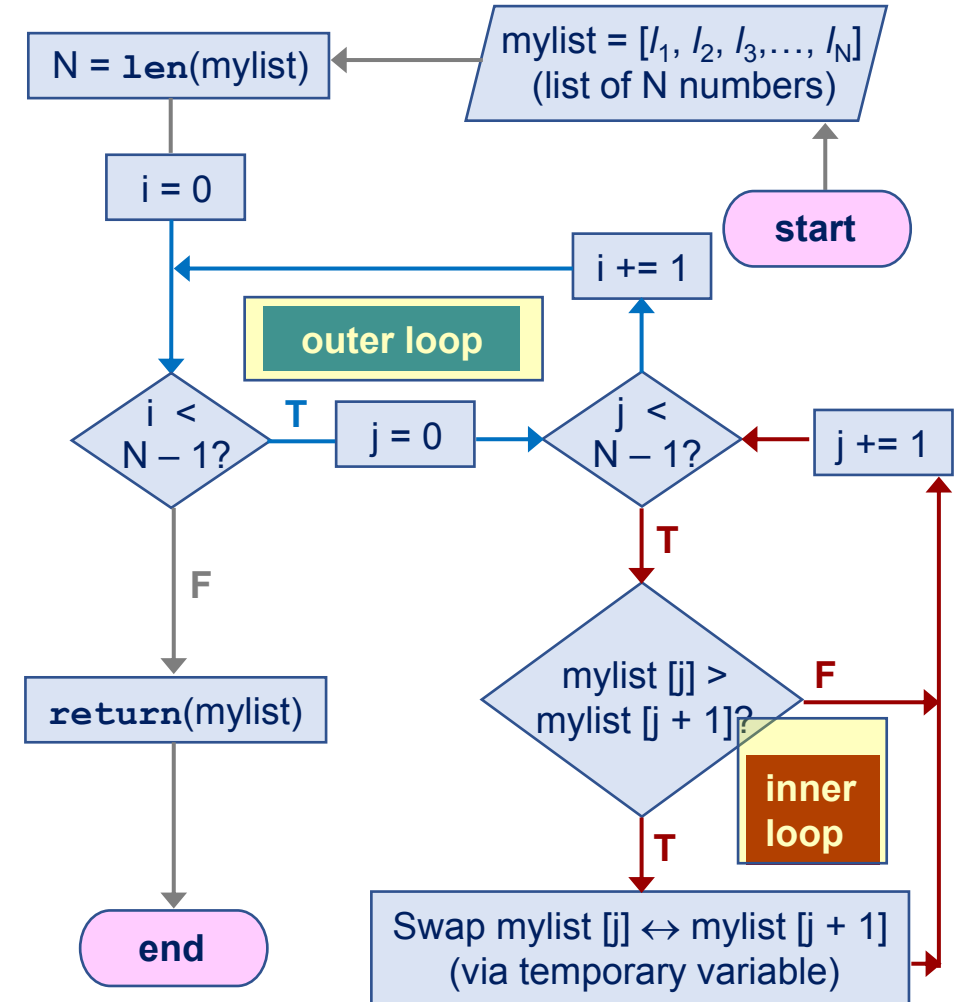
Step 2: Take the first element and compare it with the immediate neighbor to the right: $a_i > a_{i+1}$

- If true: swap and increment i by one
- If false: increment i by one

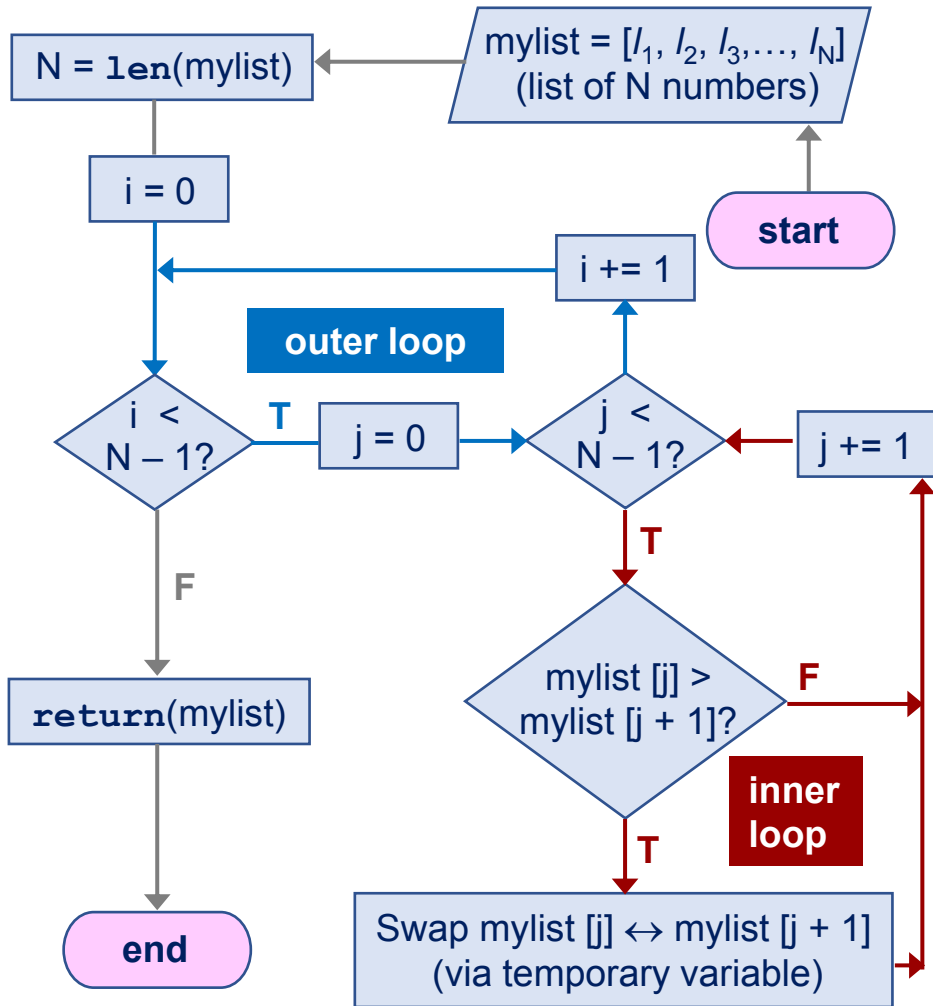
Step 3: Repeat step 2, $N - 1$ times

Pseudocode

```
while k < N
    for i = 0 to N - 1
        if a[i] > a[i + 1]
            then
                swap()
        end
    k = k + 1
end
```



Suggested Code



```
def bubbleSort(alist):
    for passnum in range(len(alist)-1):
        for i in range(len(alist)-1):
            if alist[i]>alist[i+1]:
                temp = alist[i]
                alist[i] = alist[i+1]
                alist[i+1] = temp
```

```
alist = [6,5,4,3]
bubbleSort(alist)
print(alist)
```



- Is it efficient?
- Any possible improvement?

Bubble Sort: Second Pass - End

- At the end of the first pass, there are $n - 1$ items left to sort.
- At the end of the second pass, there are $n - 2$ items left to sort.
- Number of items to be sorted decreases while the number of pass increases.



93 in place after
the first pass



Second Pass



Improved Code of Bubble Sort

```
def bubbleSort(alist):
    for passnum in range(len(alist)-1):
        for i in range(len(alist)-passnum-1):
            if alist[i]>alist[i+1]:
                temp = alist[i]
                alist[i] = alist[i+1]
                alist[i+1] = temp

alist = [54,26,93,17,77,31,44,55,20]
bubbleSort(alist)
print(alist)
```



- The inner loop executes **$n - 1$** times at first, linearly dropping to just once
- On average, inner loop executes about **$n/2$** times for each execution of the outer loop

 **Any possible improvement?**



Recall

Since each pass places the next largest value in place, the total number of passes necessary will be $n - 1$.



What is the output of the following Python code?

```
def bubbleSort(alist):  
    for passnum in range(len(alist)-1):  
        for i in range(len(alist)-passnum-1):  
            if alist[i]>alist[i+1]:  
                temp = alist[i]  
                alist[i] = alist[i+1]  
                alist[i+1] = temp  
        print("pass",passnum+1, ":",alist)  
  
alist = [1,2,3,4,5,6]  
bubbleSort(alist)  
print(alist)
```



What is the output of the following Python code?

```
def bubbleSort(alist):  
    for passnum in range(len(alist)-1):  
        for i in range(len(alist)-passnum-1):  
            if alist[i]>alist[i+1]:  
                temp = alist[i]  
                alist[i] = alist[i+1]  
                alist[i+1] = temp  
        print("pass",passnum+1, ":",alist)  
  
alist = [1,2,3,4,5,6]  
bubbleSort(alist)  
print(alist)
```

Answer

```
pass 1 : [1, 2, 3, 4, 5, 6]  
pass 2 : [1, 2, 3, 4, 5, 6]  
pass 3 : [1, 2, 3, 4, 5, 6]  
pass 4 : [1, 2, 3, 4, 5, 6]  
pass 5 : [1, 2, 3, 4, 5, 6]  
[1, 2, 3, 4, 5, 6]
```

Break: Early Exit to be More Efficient

```
procedure bubbleSort( list : array of items )
  loop = list.count;
  for i = 0 to loop-1 do:
    swapped = false
    for j = 0 to loop-i-1 do:
      /* compare the adjacent elements */
      if list[j] > list[j+1] then
        /* swap them */
        swap( list[j], list[j+1] )
        swapped = true
      end if
    end for
    /*if no number was swapped that means array is sorted now, break the loop.*/
    if(not swapped) then
      break
    end if
  end for
end procedure return list
```

- How can we know if the list is sorted?
- The pass go through the list but no swaps are needed, which indicates that the list is sorted.



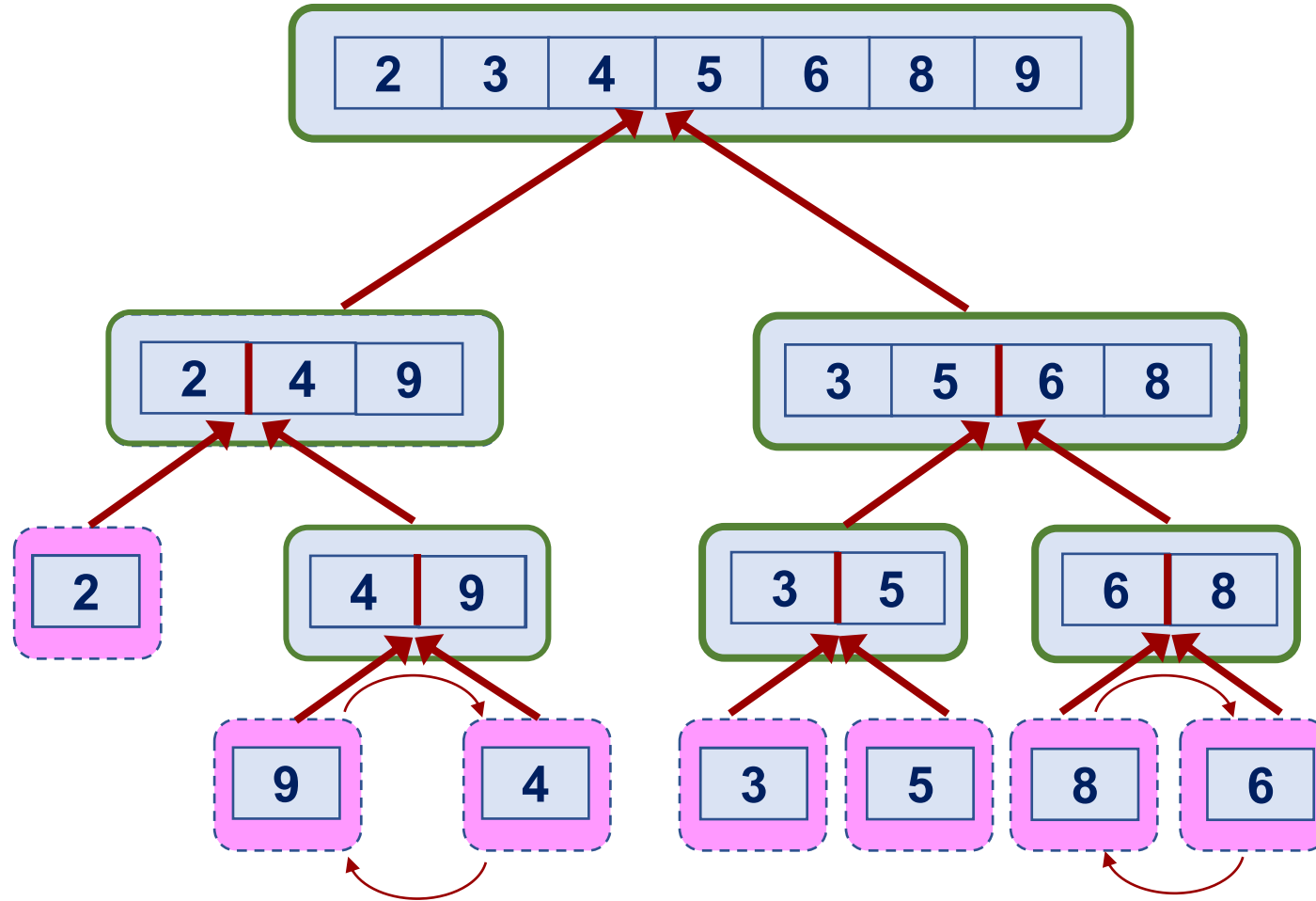
Improved Code of Bubble Sort: Run

```
def bubbleSort(alist):  
    for passnum in range(len(alist)-1):  
        swapped = False  
        for i in range(len(alist) -passnum-1):  
            if alist[i]>alist[i+1]:  
                temp = alist[i]  
                alist[i] = alist[i+1]  
                alist[i+1] = temp  
                swapped = True  
        print("pass",passnum+1, ":",alist)  
        if not swapped:  
            break;  
  
alist = [1,2,3,4,5,6]  
bubbleSort(alist)  
print(alist)
```



pass 1 : [1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5, 6]

Recall: Merge Sort - Execution Example



Step 4: Merge the two sorted subsequences to form the output sequence.

Merge Sort Function



```
def mergesort(list_of_items):  
    list_len = len(list_of_items)  
  
    # base case  
    if list_len < 2:  
        return list_of_items  
  
    left_list = list_of_items[:list_len // 2]    # //  
    right_list = list_of_items[list_len // 2:]    # "//" to force division  
  
    # merge sort left and right list recursively  
    left_list = mergesort(left_list)  
    right_list = mergesort(right_list)  
    return merge(left_list, right_list)
```

Merge Adjacent List

This means we have a list on the left and a list on the right.

Step 1

Compare the first elements of both lists one by one.

Step 2

Add this value to the list of “sorted items”. Move the smaller element out of the list that it was found in.

Step 3

Repeat the process until only a single list remains.

Step 4

One list should still contain elements. This list is sorted. Move its contents into the result list.

Suggested code

```
while left_list and right_list:
    if left_list[0] < right_list[0]:
        result_list.append(left_list[0])
        left_list.pop(0)
    else:
        result_list.append(right_list[0])
        right_list.pop(0)

if left_list:
    result_list.extend(left_list)
else:
    result_list.extend(right_list)
```

Merge Function

```
def merge(left_list, right_list):  
  
    result_list = []  
  
    # while left and right list has elements  
    while left_list and right_list:  
        if left_list[0] < right_list[0]:  
            result_list.append(left_list[0])  
            left_list.pop(0)  
        else:  
            result_list.append(right_list[0])  
            right_list.pop(0)  
  
    #left list still contain elements. Append its contents to end of the result  
    if left_list:  
        result_list.extend(left_list)  
    else:  
        #right list still contain elements. Append its contents to end of the result  
        result_list.extend(right_list)  
  
    return result_list
```

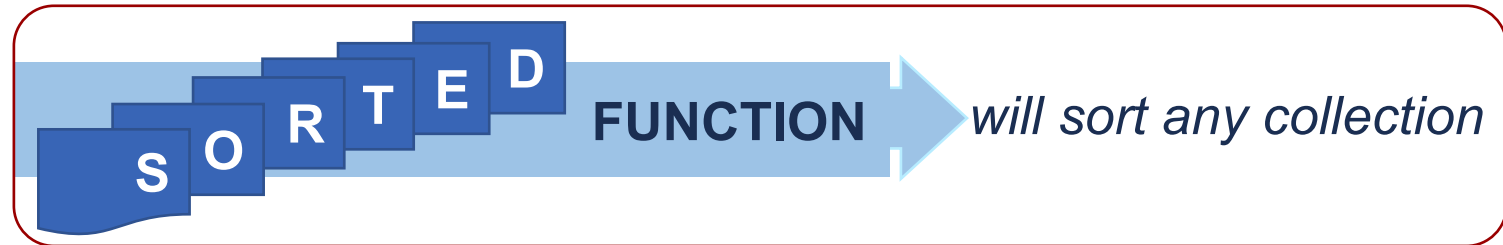


Application

```
list_test = [21, 1, 26, 45, 3, 3, 4, 27, 43, 34, 46, 40]
new_list = mergesort(list_test)
for item in new_list:
    print(item, end = ', ')
```



1, 3, 3, 4, 21, 26, 27, 34, 40, 43, 45, 46,



Bubble Sort Algorithm

Pseudocode

```
while k < N
    for i = 0 to N = 1
        if a[i] > a[i + 1]
            then
                swap()
        end
    k = k + 1
end
```

SORTING in PYTHON

Merge Sort Algorithm

Pseudocode

```
MergeSort(list, first, last)
if (first < last)
    middle = (first + last) div 2
    MergeSort(list, first, middle)
    MergeSort(list, middle+1, last)
    Merge(list, first, middle, last)
end if
```